The Complete Coding Agent, Harness, and Scaffold Compendium: Multi-Round Architectural Analysis, Documentation Index, and the OmniHarness Unified Specification

This report delivers a three-part engineering reference and system specification covering the 2026 AI software engineering landscape: (1) a modular docs/ directory structure and a compact master docs/INDEX.md mapping 65 coding agents, harnesses, and scaffolds across 12 source-code dimensions[1]; (2) a four-round chained architectural analysis of every major meta-harness, CLI agent, IDE extension, interoperability protocol, and autonomous software engineering (SWE) research scaffold—including **Google Research's Harness²**, **DeepSeek Harness (dsh)**, **Anthropic's Claude Code**, **OpenCode**, **Roo Code / Zoo Code**, **OpenAI Codex CLI**, **Pi (pi-mono)**, **Aider**, **SWE-agent / mini-SWE-agent**, **OpenHands**, **Agentless**, **AutoCodeRover / SpecRover**, **Moatless Tools / SWE-Search**, **Block/AAIF Goose**, **Nous Research Hermes Agent**, **Google Gemini CLI & Antigravity 2.0**, **Factory Droid CLI**, **Sourcegraph Amp**, **AWS Kiro**, **Plandex**, **Mentat**, **Continue**, and **Charmbracelet Crush**; and (3) the complete architectural blueprint and reference implementation for **OmniHarness (omni)**, a unified CLI and agent harness that synthesizes the highest-leverage mechanisms from every system in the corpus.

A critical empirical finding governs the architecture of OmniHarness: **static scaffolding bloat acts as a heavy cost multiplier that increases inference spend by 3x to 5x without improving task resolution.** Controlled 2026 evaluations by researchers at Shandong University (testing 5 models plus a Claude Opus 5 anchor across 492 SWE-bench Verified tasks) and UC Berkeley's Sky Computing Lab ("HarnessTax," testing 7 models across 30 benchmark tasks) indicate that holding the base LLM fixed while swapping between heavyweight harnesses (Claude Code, sending 16,581 tokens on Turn 1), mid-weight harnesses (OpenCode, sending 7,025 tokens on Turn 1), and minimalist harnesses (mini-SWE-agent at 829 tokens or Pi at under 1,000 tokens) shifts pass rates by less than run-to-run variance (~2 percentage points, against a 13% median task-flip noise floor) while multiplying per-task API spend threefold to fivefold[4]. Conversely, targeted structural mechanisms—such as SWE-agent's lint-gated Agent-Computer Interface (ACI), Aider's Personalized PageRank repository map, Roo/Zoo Code's Levenshtein fuzzy diff engine, OpenCode's real-time Language Server Protocol (LSP) diagnostic feedback, and Google Harness²'s contrastive Propose–Probe–Compose loop—produce measurable gains (+10.7 to +11.8 percentage points in author-reported evaluations) when engineered without always-on prompt bloat[6].

OmniHarness resolves this trade-off by combining a **sub-900-token micro-kernel** (from Pi

and mini-SWE-agent) with **Cordis spatiotemporal plugin composition** (from DeepSeek Harness), **3-layer progressive disclosure** (from Google Antigravity and Pi), **OS-level kernel sandboxing** (from OpenAI Codex CLI and Claude Code), and **offline recursive harness evolution** (from Google Harness[2]).

EMPIRICAL BENCHMARK & CONTEXT EVALUATION • OCTOBER 2026

The Empirical **Harness Tax**: Static Prompt Overhead Multiplies Cost While Minimalist Loops Match Accuracy

3x–5x
Inference Cost Multiplier Across Harnesses
Holding models fixed, task cost varies up to 3x (Shandong Univ.) and 5x (UC Berkeley HarnessTax) for equivalent accuracy.

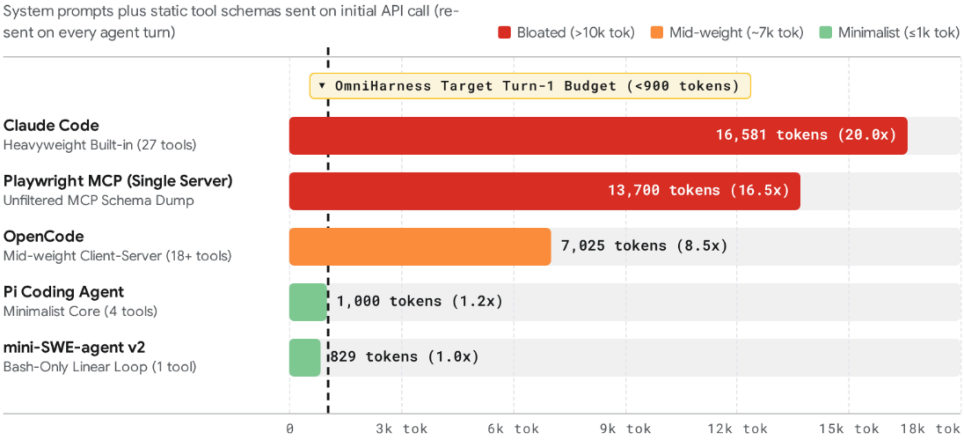
13%
Median Task-Flip Noise Floor Between Runs
Identical setups vary by 2.2–6.0 pp across runs; Claude Code & mini-SWE-agent score within 2 pp on 447 SWE-bench tasks.

+11.8 pp
Max Gain From Harness² Recursion
Sandboxed execution, AST/LSP localization, and recursive loop optimization drive accuracy—not static tool schema dumps.

PANEL A • STATIC SCAFFOLDING OVERHEAD

Turn-1 Fixed Context Token Overhead Prior to Reading Repository Files

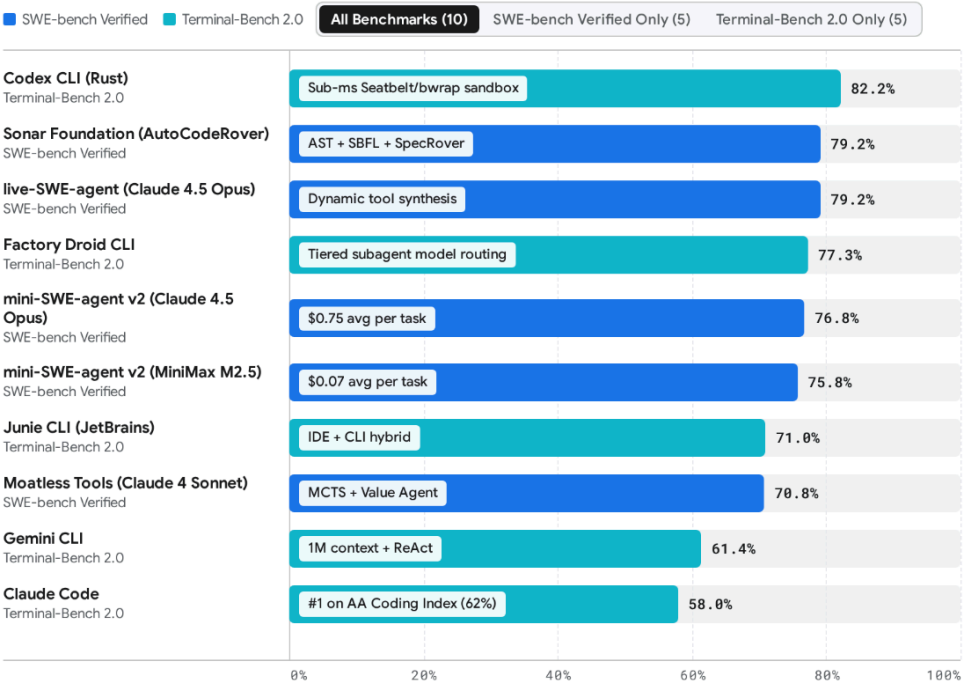
System prompts plus static tool schemas sent on initial API call (re-sent on every agent turn)



PANEL B • BENCHMARK RESOLUTION ACCURACY

Recorded Accuracy on SWE-bench Verified & Terminal-Bench 2.0 (%)

Comparing lightweight bash loops against heavyweight & specialized scaffolds



Top panel compares Turn-1 context token overhead (system prompts plus static tool schemas sent prior to reading repository files). Bottom panel displays top recorded resolution rates on SWE-bench Verified and Terminal-Bench 2.0 across agent scaffolds as of October 2026.

Data sources: Shandong University & UC Berkeley HarnessTax Studies (VentureBeat) • SWE-bench Official Leaderboards • Terminal Trove AI Coding Agents Comparison • Pi Coding Agent Architecture Research

Part 1: The docs/ Directory Architecture and Compact Master Index

To structure the architectural specifications of 65+ coding agents, meta-harnesses, protocols, and research scaffolds into a modular knowledge base, the docs/ directory is partitioned into seven subdirectories (00-taxonomy through 06-omniharness-spec). Agent-specific files in 01 through 04 focus on each system's unique control loop and design trade-offs while pointing to 05-core-subsystems/ for shared mathematical and systems primitives.

1.1 Directory Sub-Tree Layout

Directory / File Path	Scope & Architectural Contents
docs/INDEX.md	Master routing table, 12-dimension architectural matrix, and compact lookup for all 65 cataloged agents[2].
docs/00-taxonomy/01-rombaut-12-dimensions.md	Benjamin Rombaut's source-code taxonomy across Layer 1 (Control), Layer 2 (Tools/Environment), and Layer 3 (Resource Management) (arXiv:2604.03515)[1].
docs/00-taxonomy/02-empirical-harness-tax.md	Shandong University and UC Berkeley HarnessTax cost-vs.-accuracy evaluations, prompt cache economics, and noise-floor methodology[5].
docs/00-taxonomy/03-65-agent-landscape-matrix.md	Full census of 65 CLI, TUI, IDE, and cloud coding agents with Terminal-Bench 2.0, Coding Agent Index, and SWE-bench metrics[3].
docs/01-meta-harnesses/01-google-harness2.md	Google Research's harness2 (c1–c8 tree, Propose–Probe–Compose, parallel/sequential recursion, BenchmarkAdapter)[10].
docs/01-meta-harnesses/02-deepseek-harness-dsh.md	DeepSeek Harness (dsh) and the Cordis spatiotemporal plugin engine (~230 packages, event log, projections, Creator Mode, PTC)[16].

docs/01-meta-harnesses/03-hermes-agent.md	Nous Research's Hermes Agent (closed skill-learning loop, SQLite FTS5 lineage, 6 sandbox backends, single-select memory/context engines)[21].
docs/01-meta-harnesses/04-langchain-deepagents.md	LangChain's deepagents and dcode ("subtracting maximalist" 3-phase middleware stack, 4-layer context defense, delta reducer)[23].
docs/02-cli-agents/01-claude-code.md	Anthropic's Claude Code (v2.1.310, 515 modular prompts, 27 tools, 3-tier compaction, /subtask forks, AST-checked BashTool)[25].
docs/02-cli-agents/02-openai-codex-cli.md	OpenAI Codex (codex-rs 80+ Rust crates, pre-main hardening, Seatbelt/Bubblewrap/Landlock/Seccomp, apply_patch, Guardian)[29].
docs/02-cli-agents/03-opencode-and-crush.md	Anomaly's OpenCode (Bun/Hono server, 34+ built-in LSP diagnostic loop) and Charmbracelet Crush (Go Bubble Tea v2, crush serve workspaces)[8].
docs/02-cli-agents/04-pi-coding-agent.md	Earendil/badlogic's pi-mono (sub-1k token prompt, 4 wire protocols, append-only JSONL DAG, jiti extensions, differential TUI)[4].
docs/02-cli-agents/05-aider.md	Aider (Tree-sitter + Personalized PageRank RepoMap, 13 Coder edit formats, Architect/Editor split, lint/test reflection)[9].
docs/02-cli-agents/06-gemini-and-antigravity-cli.md	Google Gemini CLI & Antigravity 2.0 (agy, probe-verified compaction, 5 sandboxes, progressive skills, verification Artifacts)[40].
docs/02-cli-agents/07-factory-droid-and-amp.md	Factory Droid CLI (Custom Droids, Mission mode, tiered routing, --worktree) and Sourcegraph Amp (Orbs, oracle subagent)[45].

docs/02-cli-agents/08-goose-plandex-mentat.md	Block/AAIF Goose (Rust MCP runtime, YAML Recipes), Plandex (2M context, sandboxed plan branching), and Mentat (Ractor actors)[49].
docs/03-ide-and-protocol-agents/01-cline-roo-zoo-kilo.md	Cline, Roo Code (archived May 2026), Zoo Code (v3.90.0, Semble, DCG), and Kilo Code (Boomerang orchestrator, apply_diff, Shadow Git)[11].
docs/03-ide-and-protocol-agents/02-kiro-cursor-windsurf-continue.md	AWS Kiro (Specs requirements/design/tasks.md, .kiro/hooks, Steering), Cursor, Devin Desktop (Windsurf Cascade), and Continue[58].
docs/03-ide-and-protocol-agents/03-acp-mcp-and-rulesync.md	Zed's Agent Client Protocol (ACP) vs. Anthropic's MCP, dyoshikawa/rulesync, freeCodeCamp/agent-scaffold, and vibe-coding-toolkit[62].
docs/04-swe-research-scaffolds/01-swe-agent-and-mini.md	SWE-agent (ACI windowed viewer, lint-on-edit, SWE-ReX sentinel sessions) and mini-SWE-agent v2 (100-line bash-only loop)[67].
docs/04-swe-research-scaffolds/02-openhands.md	OpenHands (immutable EventStream, CodeActAgent, APIRemoteWorkspace, Agent Canvas, Automation Server)[2].
docs/04-swe-research-scaffolds/03-agentless-and-autocoderover.md	Agentless (3-stage localization, AST voting) and AutoCodeRover / SpecRover (AST search, SBFL, SelfFix replay)[73].
docs/04-swe-research-scaffolds/04-moatless-prometheus-dars-sweaf.md	Moatless/SWE-Search (MCTS + Value/Discriminator agents), Prometheus (LangGraph), DARS-Agent, and SWE-AF (Issue DAG + VCs)[2].
docs/05-core-subsystems/01-patch-and-diff-engines.md	Source-level comparison of Edit, apply_patch, apply_diff, replace_in_file, EditBlockCoder, and UnifiedDiffCoder[11].

docs/05-core-subsystems/02-context-compaction-and-repomap.md	PageRank RepoMap math, 3-tier compaction, headroom accounting, probe verification, and post-compaction rehydration[2].
docs/05-core-subsystems/03-os-sandboxing-and-ast-security.md	Seatbelt SBPL profiles, Bubblewrap + Landlock + Seccomp-BPF, SOCKS5 proxies, pre-main hardening, and Bash AST parsing[28].
docs/05-core-subsystems/04-subagent-and-multi-model-routing.md	Boomerang tasks, /subtask cache forks, Oracle escalation, worktree parallelism, and complexity tiering[45].
docs/06-omniharness-spec/01-architecture-and-cordis-core.md	5-plane OmniHarness architecture, Cordis service map, and zero-bloat boot sequence.
docs/06-omniharness-spec/02-tools-patching-and-intelligence.md	Hybrid RepoMap + LSP + SBFL engine and 5-stage fault-tolerant surgical patch applier.
docs/06-omniharness-spec/03-sandbox-compaction-and-evolution.md	Kernel sandbox policies, 3-tier context defense, and omni evolve (Propose–Probe–Compose + skill crystallization).

1.2 Master File: docs/INDEX.md (Compact Ecosystem & Routing Map)

The table below serves as the master docs/INDEX.md file, indexing the flagship harnesses and scaffolds in the research corpus across their implementation stack, license, control topology, signature architectural mechanism, and documentation target[2].

System / Scaffold	Org / Creator	Stack & License	Control Loop & State Topology	Signature Architectural Contribution (Adopted in OmniHarness)	Doc Path in docs/

Harness² (harness2)	Google Research	Python 3.12 (uv) • Apache-2.0	2-Level Recursive (Propose-Probe-Compose + Experience Reflector)	Optimizes 8 harness components (c1–c8) around frozen weights without ground-truth labels (R = 4k, P = 3k)[14]	01-meta-harnesses/01-google-harness2.md
DeepSeek Harness (dsh)	DeepSeek AI	TS/Node ≥24.9 + Python • MIT	Cordis Plugin Waterfall + Append-Only SessionEvent Log	"Everything is a plugin" (~230 packages), reversible ctx.effect(), deriveMessages() projections, Creator Mode, PTC[16]	01-meta-harnesses/02-deepseek-harness-dsh.md
Hermes Agent	Nous Research	Python (88%) + TS/Ink • MIT	Synchronous AIAgent Loop + SQLite FTS5 Lineage DB	Closed self-created skill learning loop (SKILL.md), 6 sandbox backends, 25+ gateway adapters[21]	01-meta-harnesses/03-hermes-agent.md
DeepAgents (dcode)	LangChain	Python / LangGraph • MIT	3-Phase Middleware Stack + Delta-Channel Reducer	4-layer automatic context defense (delta	01-meta-harnesses/04-langchain-deepagents.md

				reducer, 85% summary, disk spill preview, stale arg truncation)[2][3]	
Claude Code (v2.1.310)	Anthropic	TypeScript • Proprietary	Imperative Loop + Classifier Auto-Mode + Subagent Instances	3-layer compaction + 6-step rehydration, /subtask cache forks, Tree-sitter Bash AST security + Seatbelt/bwrap[25]	02-cli-agents/01-claude-code.md
Codex CLI (codex-rs)	OpenAI	Rust (80+ crates) + TS shim • Apache-2.0	Rust Event Loop + SQLite Rollouts + ThreadManager	Default-on Seatbelt/bwrap+Landlock+seccomp sandbox, pre-main ptrace/LD_* hardening, 4-pass apply_patch, Guardian[29]	02-cli-agents/02-openai-codex-cli.md
OpenCode (opencode)	Anomaly (ex-SST)	TS/Bun/Hono Server + Go TUI • MIT	Pub/Sub SSE Bus + SQLite Typed Event Log (12 Part Types)	Built-in 34+ LSP server auto-management feeding textDocument/didChange type	02-cli-agents/03-opencode-and-crush.md

				errors back to LLM[2]	
Crush (crush)	Charmbracelet	Go + Bubble Tea v2 + SQLite • FSL-1.1-MIT	Elm Architecture (Init/Update/View) + Shared CWD Workspaces	Cell-based diff TUI, embedded cross-platform Bash interpreter (crushrc), crush serve multi-client workspaces [3]	02-cli-agents/03-opencode-and-crush.md
Pi (pi-mono)	Earendil / Mario Zechner	TypeScript Monorepo • MIT	796-line while Loop (Zero try/catch) + JSONL DAG Tree	<1,000-token prompt, 4 wire protocols (pi-ai), cross-provider <thinking> handoff, /tree branching, differential TUI[4]	02-cli-agents/04-pi-coding-agent.md
Aider	Paul Gauthier	Python • Apache-2.0	Iterative Prompt-Edit-Lint-Test Loop + Destructive Summary	Tree-sitter + Personalized PageRank RepoMap (binary search token fit) + 13 model-tuned Coder edit formats[2]	02-cli-agents/05-aider.md

Gemini CLI	Google	TypeScript • Apache-2.0	ReAct Loop (AgentProtocol) + Filtered History View	1M context, hierarchical GEMINI.md, **Probe-verified LLM summarization**, 5 sandbox backends (incl. gVisor runsc)[2]	02-cli-agents/06-gemini-and-antigravity-cli.md
Antigravity 2.0 (agy)	Google	Go / Proprietary	Dynamic Multi-Agent Orchestrator + Persistent Linux Env	3-layer progressive skill disclosure, dynamic parallel subagent synthesis (/goal), verification Artifacts[3]	02-cli-agents/06-gemini-and-antigravity-cli.md
Factory Droid (droid)	Factory AI	TS / Bun SEA • Source-Avail / Prop	Spec-First / Mission Loop + Complexity-Routed Subagents	Custom Droids (.md), Light/Medium/Heavy complexity routing, Worker vs Validator model split, --worktree fan-out[47]	02-cli-agents/07-factory-droid-and-amp.md
Amp	Sourcegraph	TypeScript • Proprietary	Main Agent + Cloud Orbs + Read-Only oracle	High-reasoning oracle consultation tool (o3/GPT-5/	02-cli-agents/07-factory-droid-and-amp.md

			Subagent	GPT-6 Astra) + remote cloud Orbs with live preview portals[45]	
Goose (goose)	AAIF / Block	Rust • Apache-2.0	Rust Agent Loop + TasksManager Subagents + ACP Server	Parameterized portable YAML Recipes & sub_recipes (sequential_when_repeated), 70+ MCP extensions, native ACP[51]	02-cli-agents/08-goose-plandex-mentat.md
Plandex	Plandex AI	Go Server + LiteLLM + PG • MIT	Server-Backed Stateful Plan Loop + Sandbox Diff Staging	Branchable/rewindable cumulative multi-file plan version control before touching disk; 2M token tree-sitter maps[49]	02-cli-agents/08-goose-plandex-mentat.md
Cline (cline)	Cline Bot Inc.	TypeScript • Apache-2.0	Recursive Loop + Plan/Act Mode Split + condense Tool	Order-invariant multi-diff apply algorithm, model-specific markers (---/+++ vs	03-ide-and-protocol-agents/01-cline-roo-zoo-kilo.md

				>>>/<<<), LLM-initiated condense[2]	
Roo Code → Zoo Code	Zoo Code Org (ex-Roo)	TypeScript VS Code / JB • Apache-2.0	Mode-Switched Loop + 🪃 Boomerang Subtask Orchestrator	Boomerang new_task/attempt_completion isolation, :start_line: Levenshtein apply_diff, Shadow Git, Semble, DCG[11]	03-ide-and-protocol-agents/01-cline-roo-zoo-kilo.md
AWS Kiro	Amazon Web Services	VS Code Fork + Rust CLI • Prop	Spec-Driven Pipeline + Event-Driven Background Hooks	Executable Specs (requirements.md, design.md, tasks.md), .kiro/hooks on file events, property-based testing[60]	03-ide-and-protocol-agents/02-kiro-cursor-windsurf-continue.md
SWE-agent & SWE-ReX	Princeton / Stanford	Python • MIT	DefaultAgent Loop + 7 Composable History Processors	Agent-Computer Interface (100-line viewer, flake8 lint-on-edit auto-revert) + SWE-ReX sentinel shell	04-swe-research-scaffolds/01-swe-agent-and-mini.md

				sessions[2]	
mini-SWE-agent v2	Princeton / Stanford	Python (~100 lines) • MIT	Stateless subprocess.run Loop + Linear History + Exceptions	Zero tool-calling schema (bash only), trajectory === prompt messages, InterruptAgentFlow control exceptions[2]	04-swe-research-scaffolds/01-swe-agent-and-mini.md
OpenHands	OpenHands AI	Python + React/TS Canvas • MIT	Event-Sourced EventStream + CodeActAgent	Immutable EventStream with compression views, stateful pexpect bash + BrowserGym, APIRemoteWorkspace[2]	04-swe-research-scaffolds/02-openhands.md
Agentless	UIUC / OpenAutoCoder	Python (~700 lines) • MIT	Fixed 3-Phase Pipeline (Localize ⟶ Repair ⟶ Validate)	3-stage hierarchical localization (File ⟶ Skeleton ⟶ Line) + reproducer test synthesis + AST majority voting[73]	04-swe-research-scaffolds/03-agentless-and-autocoderover.md
AutoCode Rover /	NUS / SonarSourc	Python + Tree-sitter •	Generator Coroutine	Stratified AST search	04-swe-research-scaff

SpecRover	e	Prop/OS	(ActionState) + SelfFix Replay	+ Spectrum-Based Fault Localization (SBFL) + SelfFix upstream failure attribution & replay[74]	olds/03-agentless-and-autocoderover.md
Moatless / SWE-Search	Albert Örwall et al.	Python • MIT	Monte Carlo Tree Search (MCTS) over File Context Snapshots	Modified UCT with depth bonus + Value Agent hindsight feedback (ϵ_t) + multi-agent Discriminator debate[2]	04-swe-research-scaffolds/04-moatless-prometheus-dars-sweaf.md
SWE-AF	Agent-Field	Python • Open Source	5-Agent Planning Chain (Issue DAG) + 3 Nested Loops	Runtime Issue DAG mutation (FAILED_NEEDS_SPLIT), typed incompleteness gaps, signed Verifiable Credentials [76]	04-swe-research-scaffolds/04-moatless-prometheus-dars-sweaf.md

Part 2: Chained Deep-Research Rounds Across All Harnesses and Scaffolds

Round 1: Meta-Harnesses and Self-Improving Agent Frameworks (docs/01-meta-harnesses/)

The defining architectural advance of 2026 is the decoupling of the **agent execution**

harness—the runtime operating system of prompts, tools, memory, guardrails, subagents, skills, scripts, and plugins surrounding a base model—from the **harnessing optimizer** that composes or rewrites those components across tasks[6]. Google Research's **Harness²** and DeepSeek AI's **DeepSeek Harness (dsh)** solve orthogonal halves of this meta-harness problem: Harness² is an outer-loop self-evolution framework that edits an agent's workspace playbook (c1–c8) via behavioral contrast across task rollouts[10], whereas DeepSeek Harness is an inner-loop composable runtime where every subsystem—including the agent loop, tool pipeline, and session log—is mounted as a hot-swappable **Cordis** plugin[18]. In a unified system, Harness²'s outer recursive optimizer can directly target DeepSeek Harness's Cordis plugin tree as its editable substrate.

1.1 Google Research's Harness² (google-research/harness2, arXiv:2610.14224)

Released in October 2026 under the Apache 2.0 license, **Harness²** formalizes *recursive agent harnessing in an open world*, enabling an AI agent to adapt its surrounding system to unfamiliar repositories, legal documents, or professional workflows without retraining the underlying language model's weights[6].

Harness² treats both the base model weights and the platform's root system prompt as frozen, exposing an 8-component editable workspace tree (c1 through c8) grouped by runtime function[14]. Under **Context Construction**, **c1 System Prompt** defines global operating rules and execution priorities (such as rules in AGENTS.md), **c2 Guardrails** enforces permitted behavior and operational boundaries, **c3 Memory** manages persistent task state, daily notes, and lesson-learned protocols, and **c6 Skills** stores reusable domain procedures (SKILL.md)[10]. Under **Tool Interface**, **c4 Tool Guidance** specifies heuristics for selecting tools and **c7 Scripts** provides executable helper scripts bundled inside skills[10]. Under **Delegation**, **c5 Subagents** configures specialist subagent briefs and orchestration rules, while under **Middleware**, **c8 Plugins / Hooks** defines scheduled checks, validation gates, and interceptors inside the tool loop[14].

When an agent encounters an unseen task in production, no ground-truth verifier or answer rubric exists to score intermediate attempts[6]. Harness² addresses this at the **task level** through a contrastive, verifier-free **Propose–Probe–Compose** step[14]. During **Propose**, an executor runs the initial harness on the task, and a harness editor proposes invariant (general) and domain-specific edits to c1–c8[10]. During **Probe**, candidate probes execute these proposed harness variations on the current task[10]. During **Compose**, a harness composer inspects the *behavioral contrasts* between execution trajectories (such as avoided tool failures, cleaner verification steps, or completed deliverables) and synthesizes a refined harness without ground-truth feedback[6]. Task-level recursion scales via the --k K budget flag in two modes: **Parallel Recursion (--mode parallel)** probes K general and K domain-specific candidates simultaneously and produces K compositions from shared execution evidence (exploring wider), whereas **Sequential Recursion (--mode sequential)** passes the refined harness and accumulated evidence through K successive Propose–Probe–Compose cycles (refining deeper)[10]. Across every task and mode, the framework enforces a strict resource identity of R = 4k rollouts and P = 3k improver sessions[15]. At the **harnessing level** across tasks, an

Experience Reflector retains adopted harness edits and evidence-backed lessons (with scope, verification date, and invalidation triggers) in a persistent *improvement harness* library to guide future tasks[14].

Harness² executes runs using **OpenCode (oc, default)** or **OpenAI Codex (--substrate cx)** as its underlying substrate[14]. Custom domains implement an 11-method BenchmarkAdapter protocol in harness2/core/spec.py (including execute(), delivered(), trajectory(), validate_tree(), and leak_globs to block rubric files from reaching the proposer)[15]. Before any run begins, validate_harness.py enforces three fail-closed validation gates: (1) harness2.core.tree.validate(), which rejects untriggered skills, frontmatter mismatches, missing scripts, over-cap always-on text, and malformed hooks; (2) the domain adapter gate validate_tree(); and (3) a Grader-Verdict Leak Tripwire that aborts if rubric markers appear in candidate trees[15]. Candidate scripts, plugins, and hooks are read strictly as static text during optimization and are never executed by the improver[15].

In Google Research's benchmark evaluations across four model–harness configurations using GLM-5 and Gemini 3.5 Flash, a single recursive step (--k 1) improved scores by up to **+11.5 points** on **WorkBuddy-Bench** (office, web, and code subsets inside Docker), **+4.5 points** on **JobBench** (professional tasks), and **+11.8 percentage points** on **LAB** (legal tasks partial pass rate)[10]. The authors note two operational caveats: first, the reported gains come from their own harness and task setup, so teams should run independent evaluations before adopting evolved harnesses[6]; second, upstream executors run with unrestricted network access and Codex's sandbox disabled by default, requiring explicit network/host blocks so agents do not fetch public benchmark rubrics (RUBRICS.json) online[14]. Because recursive optimization incurs additional rollout spend (for example, 6 extra optimization runs at $0.20 per task adds $1.20 in overhead, requiring 30 subsequent tasks at $0.04 savings per task to break even), best practice is to freeze all surfaces except a single skill or subagent brief during a bounded trial and enforce hard spend caps (--cap-usd)[10].

1.2 DeepSeek Harness (deepseek-ai/deepseek-harness, dsh)

Open-sourced under the MIT license by DeepSeek AI (gaining 165,000 GitHub stars in its first week according to a MindStudio review and surpassing 261,000 stars on GitHub by October 2026), **DeepSeek Harness (dsh)** is a TypeScript/Node.js (≥24.9) and Python agent runtime built on the rule that **"Everything is a plugin"**[16]. Where Claude Code, Codex, and OpenCode keep their lower-level agent loops and session stores fixed, dsh structures ~230 workspace packages (@deepseek-ai/dsh-*) over a vendored dependency-injection kernel called **Cordis**[17]. Described in the paper *A Programming Paradigm for Spatiotemporal Composability*, Cordis manages plugin lifecycles across two axes[18]. Through **spatial composability**, plugins never import concrete provider implementations; they resolve capabilities through stable context keys (ctx.sessions, ctx.systemPrompt, ctx.tools, ctx.agents, ctx.agentLoop, ctx.llm, ctx.jobs) and declare required dependencies via export const inject = ["tools", ...][18]. Cordis waits until all declared services exist before activating a plugin and automatically deactivates dependents if a required service disappears[18]. Through **temporal composability**, every registration—whether an event listener, prompt section, tool schema, or UI slot—is tracked as a reversible side-effect

via ctx.effect() or ctx.on() that returns a disposer[17]. Unloading or hot-reloading a plugin cleanly unwinds its registrations without leaving orphaned listeners[18]. Every subsystem is factored into a three-role **capability seam**—a **Service Definition** (interface), a **Service Provider** (implementation), and a **Consumer** (typically a model-facing tool)—allowing filesystems (core/fs), shells (core/shell), persistent terminals (core/terminal), SSH (core/ssh), LSPs (core/lsp), sandboxes (core/sandbox), and the agent loop itself (core/agent-loop) to be replaced from configuration[17].

A running dsh instance is composed at boot from a five-layer configuration hierarchy: *Empty root → Profile bundles (dsh-base, web, headless, sdk, sdk-minimal, acp, desktop) → Profile cordis.patch.yml → Harness home cordis.patch.yml → Command-line --patch overlays*[19]. Hardcoded DEFAULT_* constants are prohibited in favor of validated Config fields in cordis.yml, which fails fast at load time on missing references[17]. Inside the runtime, dsh distinguishes between a **Step** (one LLM call plus the tools it invokes) and a **Turn** (zero or more steps, opening before input is claimed and closing when nothing further is owed)[18]. Events are partitioned into three domains: **Session Events (session/event)** are durable facts written to the append-only log (turn/start, step/start, system/message, user/message, assistant/message, assistant/attempt, tool/call, tool/result, compaction/*, turn/end); **Agent Events (agent/*)** carry live in-flight Agent instances through waterfall hooks (agent/pre-step, agent/request, agent/assistant-stream, agent/turn-stopping) where a listener can rewrite input or short-circuit execution by returning without calling next(); and **Capability Events** attach policy to subsystem seams (fs/*, tools/pre-execute → tools/execute → tools/post-execute, telemetry/*)[17].

dsh enforces a strict session invariant: **"Model-visible means logged"**[19]. Anything sent to an LLM request must be deterministically reconstructable from the append-only SessionEvent log via deriveMessages()[17]. The dsh-session-projection service (ctx.sessionProjections) folds committed events incrementally into pure ProjectionDefinition views (snapshot()), enabling Trajectory inspection, replay, turn-boundary session forking (ctx.agents.create({ sessionId, seed, inheritedEventCount })), and identical loop projections across both the TypeScript and Python SDKs[17]. Beyond **Standard** and **Minimal** modes, dsh ships **PTC (Programmatic Tool Calling)** via packages/core/ptc-runtime/ (executing multi-tool scripts inside a confined sandbox with typed binding namespaces and PtcRunFailure taxonomy) and **Creator Mode** (packages/core/extensions/), where the agent loads skills/cordis-plugin-development/SKILL.md, writes a new plugin (package.json + client/server JS) from a plain-language chat request, and mounts it live into the current session[16]. DeepSeek notes in its safety documentation that dsh remains in developer preview with breaking API changes and has not yet undergone a formal security audit[18].

1.3 Nous Research's Hermes Agent & LangChain's DeepAgents

Two additional open-source harnesses demonstrate how agent state and middleware evolve over extended tasks:

- **Hermes Agent (NousResearch/hermes-agent, 257k+ GitHub stars, MIT)**: Implemented in Python (88%) and TypeScript/Ink (8.8%), Hermes centers on a **closed skill-learning loop**[21]. When its synchronous AIAgent loop (run_agent.py,

agent/conversation_loop.py) solves a complex problem, it extracts the procedure into a reusable SKILL.md file (following the agentskills.io standard) under ~/.hermes/skills/, which is auto-discovered and refined in future sessions[21]. Its model_tools.py facade and tools/registry.py auto-register 70+ tools across ~28 toolsets at import time and execute terminal commands across **6 built-in sandbox backends** (local, docker, ssh, modal, daytona, vercel_sandbox, plus Singularity via plugin)[22]. Session history is stored in SQLite (hermes_state.py) with **FTS5 full-text search** and parent/child lineage tracking across context compressions, while a long-running **Messaging Gateway** routes sessions across 25+ platform adapters (CLI, Desktop, Telegram, Discord, Slack, WhatsApp, Signal) and cron jobs[22].

- **LangChain DeepAgents (deepagents & dcode, MIT)**: Examined alongside pi and dsh in an August 2026 source-level multi-case study (arXiv:2608.23953), deepagents illustrates the **"subtracting maximalist"** trajectory[23]. Built as a three-phase middleware stack over LangGraph (graph.py:816-893), it initially shipped with an authored base prompt and default todo/planning middleware, but later deprecated the base prompt, stripped built-in tool-usage prose (PR #4859), and removed default todo/planning middleware (PR #4929) after finding that explicit planning middleware should be an opt-in per-model profile rather than a default token tax[24]. Its primary architectural contribution is its **4-layer automatic context defense**: (1) a delta-channel reducer replacing LangGraph's quadratic whole-list checkpointing, (2) cheap truncation of stale tool-call arguments first, (3) eviction of oversized tool outputs to storage behind a head/tail preview and file pointer, and (4) LLM summarization near 85% of the context window[23].

Round 2: Production Terminal & CLI Coding Agents (docs/02-cli-agents/)

2.1 Anthropic's Claude Code (v2.1.310)

Claude Code is Anthropic's closed-source TypeScript terminal and IDE coding agent, ranking #1 on the Artificial Analysis Coding Agent Index (62%) and scoring 58.0% on Terminal-Bench 2.0[3]. Prompt-tracking analysis (Piebald-AI/claude-code-system-prompts) shows that as of v2.1.310 (October 23, 2026), Claude Code assembles its prompt dynamically from **515 modular prompt pieces**—including conditional environment prompts, 27 built-in tool descriptions (Read, Edit, Write, Bash, Glob, Grep, TodoWrite, Agent, ReportFindings, ScheduleWakeup, ClaudeDesign, and 8 Chrome MCP tools), subagent prompts, and /init onboarding skills—which users can customize locally via tweakcc[25].

To sustain long sessions, Claude Code manages context through a **three-layer compaction system** paired with a **six-step post-compaction rehydration sequence**[25]. First, **Microcompaction** continuously splits tool outputs (Read, Bash, Grep, Glob, WebSearch, WebFetch, Edit, Write) into a recent inline "hot tail" and disk-backed "cold storage" referenced by file path[25]. Second, **Auto-Compaction** monitors free context space against reserved *Output Headroom* (so responses never truncate) and *Compaction Headroom* (so the summarization call itself has room to complete)[25]. Third, **Manual Compaction (/compact [focus_hint])** allows users to compress context cleanly at task boundaries with custom focus

directives[25]. Because repeated summarization alone can cause severe decision loss over very long sessions (users report losing 12 of 12 early architectural decisions after 60+ compactions[105]), Claude Code immediately follows summarization with a six-step rehydration sequence: (1) insert a **Boundary Marker**, (2) inject the structured **Summary Message** with an automatic continuation instruction, (3) **re-read the 5 most recently accessed files** into context, (4) restore the **Todo List**, (5) restore **Plan State**, and (6) restore startup **Hook Outputs**[25].

For delegation, the Agent tool (renamed from Task in v2.1.63) spawns isolated subagent instances up to a configurable nesting depth (CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH, up to depth 3–5) and concurrency cap (CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS, default 20)[26]. Built-in subagents include **Explore** (an 862-token read-only codebase search agent denied Write and Edit that skips CLAUDE.md and git status across quick, medium, or very thorough modes), **Plan** (a 1,066-token read-only planning researcher), and **General-purpose**[26]. Custom subagents are defined in .claude/agents/<name>.md with YAML frontmatter (name, description, tools, model, skills) and enforced by SubagentStop hooks[26]. When a delegated subtask requires the parent's full conversation context, /subtask creates a **Fork** that inherits the parent's exact message history and system prompt, reusing the provider's **prompt cache** so the subtask avoids paying re-prefill token costs[26].

For shell execution, BashTool (src/tools/BashTool/BashTool.ts) implements a multi-layered security pipeline[28]. Before checking command exclusions (excludedCommands), shouldUseSandbox.ts runs a **safety fixed-point reduction** that iteratively strips environment variables and command wrappers (such as timeout) to identify the underlying base command[28]. Next, parseCommandRaw and parseForSecurityFromAst (bashPermissions.ts) parse the command into a **Tree-sitter AST** to extract all subcommands, operators, and redirections, while validatePath and isDangerousRemovalPath block out-of-bounds paths and destructive removals[28]. When sandboxing is enabled (/sandbox or @anthropic-ai/sandbox-runtime), commands execute inside macOS **Seatbelt** (~1–2% CPU overhead) or Linux/WSL2 **bubblewrap (bwrap) + seccomp-bpf + socat** (<10ms startup, ~2–3% CPU), restricting writes to CWD and routing network traffic through a SOCKS5/HTTP proxy filtered by allowedDomains[79]. Because Seatbelt profiles require literal paths rather than **/.env* globs, protecting project .env files and preventing unsandboxed retries requires pairing sandbox.credentials.files and "allowUnsandboxedCommands": false with a PreToolUse hook on Bash[79].

2.2 OpenAI Codex CLI (openai/codex, codex-rs)

Open-sourced under the Apache 2.0 license, OpenAI's **Codex CLI** achieves the highest accuracy among evaluated coding agents on Terminal-Bench 2.0 (**82.2%**, Rank #4) and scores 62% on the Coding Agent Index[3]. While launched via a thin Node.js shim (codex-cli/bin/codex.js), the entire implementation resides in a native **Rust workspace (codex-rs/) of 80+ crates**, including cli, tui (ratatui + crossterm), core, exec, app-server (WebSocket daemon for IDE extensions and SDKs), sandboxing, linux-sandbox,

process-hardening, rollout, and guardian[29].
Codex introduces four standout systems engineering mechanisms:

1. **Pre-Main Process Hardening (codex-process-hardening)**: Executed automatically before main() via the #[ctor::ctor] attribute, Codex hardens its own process against local credential extraction and code injection[29]. On macOS, it calls libc::ptrace(libc::PT_DENY_ATTACH, ...) to block debugger attachment, sets RLIMIT_CORE to 0 to disable core dumps, and removes all DYLD_* environment variables[29]. On Linux, it calls libc::prctl(libc::PR_SET_DUMPABLE, 0, ...), zeroes RLIMIT_CORE, and scrubs all LD_* preload variables[29].
2. **Default-On Kernel Sandboxing with .git and .codex Write Locks**: Codex enables OS-level sandboxing by default across ReadOnly, WorkspaceWrite, and DangerFullAccess policies[13]. On macOS (/usr/bin/sandbox-exec), its parameterized Seatbelt SBPL policy grants write access to WRITABLE_ROOT_0 in WorkspaceWrite mode but injects explicit (require-not (subpath (param "WRITABLE_ROOT_0_RO_0"))) exclusions to **block the agent from modifying .git/ or .codex/**, preventing malicious repos or runaway commands from tampering with Git hooks or sandbox settings[29]. On Linux, codex-linux-sandbox combines vendored **Bubblewrap (bwrap)** namespaces with **Landlock LSM** (landlock v0.4.4) path rules, no_new_privs, and **seccomp-bpf** (seccompiler v0.5.0) syscall filtering; on Windows 11 (24H2/25H2), it uses native **MXC process isolation** and Restricted Tokens[29].
3. **Four-Pass Lenient Patch Applier (apply_patch)**: Inside codex-rs/apply-patch (file_update.rs and seek_sequence.rs), Codex avoids error-prone line numbers in favor of context-anchored patch envelopes (*** Begin Patch, *** Update File: <path>, @@ <context_anchor>, -old, +new, *** End Patch)[31]. After locating the @@ anchor, seek_sequence.rs searches forward using four progressively forgiving passes: (1) exact match, (2) ignore trailing whitespace, (3) trim leading and trailing whitespace, and (4) normalize Unicode punctuation (such as dashes and smart quotes) to ASCII[31].
4. **Deferred Tool Discovery, Guardian Review, and Rollout Persistence**: To reduce context token overhead, Codex supports marking tools as **deferred** so their schemas are omitted from the initial context window and discovered on demand via a search call[110], rebuilding active tools per sampling request via built_tools() (codex.rs:6156-6164)[2]. The **Guardian subagent** (codex-rs/guardian) evaluates transcripts and proposed tool calls against a risk taxonomy to block data exfiltration, while codex-rs/rollout records every session as a structured SQLite/log **Rollout** for replay and auditing[29].

2.3 OpenCode (anomalyco/opencode) and Charmbracelet Crush (charmbracelet/crush)

The "OpenCode" ecosystem branched into two complementary open-source projects in 2025–2026[33]:

- **Anomaly's OpenCode (opencode-ai/opencode / anomalyco/opencode, ~189k–202k GitHub stars, MIT)**: OpenCode separates its presentation clients (a Go TUI, a Tauri desktop app, IDE extensions, and opencode web) from a local or remote **TypeScript/Bun**

HTTP server built on Hono[8]. Clients communicate with the server via an OpenAPI 3.1 SDK (@opencode-ai/sdk generated by Stainless) and receive real-time execution events over **Server-Sent Events (SSE)** backed by an internal publish-subscribe event bus (packages/opencode/src/bus/)[2]. Session state is persisted in SQLite via Drizzle ORM as a typed event log with **12 mutable part types** (message-v2.ts), and two-stage auto-compaction triggers when token usage exceeds 0.9 × (contextLimit - outputLimit)[2]. OpenCode's signature contribution is its **built-in Language Server Protocol (LSP) diagnostic loop** covering 34 built-in server configurations (~40 languages, including gopls, pyright, rust-analyzer, tsserver, clangd, jdtls, eslint, oxlint, and zls) over STDIO[8]. When the agent edits a file via EditTool, OpenCode sends textDocument/didChange to the matching LSP server, captures the resulting compiler and type diagnostics from its global diagnostics map, and injects them directly back into the LLM's tool observation so the model can fix broken call sites immediately[8]. OpenCode's documentation notes that LSP integration is disabled by default ("lsp": true to enable) because language servers can consume 200–500 MB of extra RAM or fall out of sync on some projects[33]. Furthermore, the Shandong University benchmark identified a crucial runtime failure mode in OpenCode: on models like Qwen3.8-27B, OpenCode trailed Claude Code by up to 9 percentage points because half of its failed runs hit a 32,000-token output cap mid-step and halted, whereas Claude Code automatically prompted the model to continue[5].

- **Charmbracelet Crush (charmbracelet/crush, FSL-1.1-MIT)**: Continuing the original Go TUI codebase under Charmbracelet, Crush is built on **Bubble Tea v2** (charm.land/bubbletea/v2), Lip Gloss, and Glamour using the functional Elm Architecture (Init() Cmd, Update(Msg) (Model, Cmd), View() View) and a cell-based diff renderer (cursed_renderer.go)[3]. Crush embeds a native, cross-platform Bash interpreter in Go so crushrc configuration scripts run identically on Windows, macOS, and Linux, syncs model metadata from the open-source **Catwalk** catalog, refreshes expired cloud tokens mid-session via aws_auth_refresh, and provides **crush serve shared workspaces**, where multiple TUI clients targeting the same --cwd share live conversation streams (AttachedClients > 0), permission queues, and LSP/MCP state under a "first-wins" security flag rule[35].

2.4 Pi Coding Agent (badlogic/pi-mono / earendil-works/pi)

Created by Mario Zechner (@badlogic, creator of libGDX) and reaching 95,000+ GitHub stars (serving as the engine beneath OpenClaw's 180k+ star project), **Pi** is the archetype of **radical minimalism and context engineering**[4].

Pi ships with a ~150-word system prompt (under 1,000 tokens total) and exactly **4 core tools** defined with TypeBox + AJV: read (supporting line-numbered text, base64 images, directory trees, and globs), write, edit (surgical search/replace returning a unified diff), and bash (30s default timeout)[4]. Its core loop (agent-loop.ts, 796 lines) contains zero try/catch blocks, no max-step limits, and no built-in MCP client[4]. Zechner omits MCP from the core because popular MCP servers dump massive tool catalogs into context on every session (citing Playwright MCP at 21 tools and 13,700 tokens), whereas **Agent Skills (SKILL.md)** and CLI tools

with READMEs (badlogic/pi-skills) use **progressive disclosure**—exposing only skill names and short descriptions at startup and reading full instructions on demand[4].
Beneath this small surface, Pi's monorepo (@earendil-works/pi-ai, pi-agent-core, pi-tui, pi-coding-agent, pi-telemetry) implements three foundational mechanisms[24]:

- **Four-Wire-Protocol Normalization & <thinking> Handoff (pi-ai)**: Rather than writing vendor-specific clients for every provider, pi-ai maps 300+ models across 30+ API providers and 3 subscription OAuth routes (Claude Pro/Max, ChatGPT Plus/Pro, GitHub Copilot) onto four wire protocols: OpenAI Completions, OpenAI Responses, Anthropic Messages, and Google Generative AI[36]. When a user switches models mid-session, pi-ai translates provider-specific reasoning traces into <thinking>-tagged text blocks so the new model continues without losing reasoning context[37].
- **Append-Only JSONL Session DAG & Crash-Recovery Register**: Sessions are stored in ~/.pi/agent/sessions/--<path>--/<timestamp>_<uuid>.jsonl where every line after the header carries an 8-character id and parentId, forming an **append-only Directed Acyclic Graph (DAG)** inside a single file alongside a single-register durable program counter for crash recovery[24]. buildContextEntries() walks from the active leaf to the root honoring compaction boundaries (which trigger at contextWindow - 16,384 tokens while keeping the most recent 20,000 tokens verbatim), and /tree, /fork, and /clone allow users to branch and navigate conversation history non-destructively[37].
- **Scrollback-Preserving Differential TUI & jiti Live Extensions**: pi-tui renders directly to the terminal's normal scrollback buffer rather than an alternate screen buffer, redrawing only changed lines inside synchronized-output escape sequences (preserving native terminal scrollback and selection, though very large sessions incur higher CPU line-comparison work)[37]. TypeScript extensions load dynamically via jiti (/reload) across 20+ lifecycle hooks (session_before_compact, session_before_tree, and pi.sendMessage() with steer, followUp, and nextTurn queues)[4].

2.5 Aider (Aider-AI/aider)

Paul Gauthier's **Aider** contributed two foundational algorithms to open-source coding agents: the **Tree-sitter + Personalized PageRank Repository Map** and **Model-Adaptive Edit Formats**[9].
To provide global repository context within a tight token budget (defaulting to max(1024, min(max_input_tokens / 8, 4096)) tokens, or 8x when no files are currently in chat), Aider's RepoMap executes a five-stage pipeline[9]. First, it parses source files using language-specific **Tree-sitter** queries (aider/queries/*.scm) to extract @name.definition.X and @name.reference.X tuples Tag(rel_fname, fname, line, name, kind) (falling back to Pygments Token.Name tokenization when queries only capture definitions, as in C++), caching tags in SQLite (diskcache.Cache) keyed by file mtime[9]. Second, it builds a directed multigraph (networkx.MultiDiGraph) where nodes are files and edges (referencer -> definer) carry heuristic weights: starting at mul = 1.0, multiplied by 10 if the identifier is mentioned by the user, by 10 if it is a meaningful snake_case/camelCase identifier (≥8 chars), by 0.1 if private (starts with _), by 0.1 if defined in more than 5 files, and by **50** if the reference originates from a file

currently active in chat_fnames, damped by the square root of reference count[9]:

$$W(\text{referencer}, \text{definer}, \text{ident}) = \text{mul}(\text{ident}, \text{referencer}) \times \sqrt{N_{\text{refs}}}$$

Third, it runs Personalized PageRank (nx.pagerank(G, weight="weight", personalization=personalize, dangling=personalize)) biased toward active chat files (100 / len(fnames))[9]. Fourth, it distributes each file's PageRank score across its outgoing (dst_file, ident) definitions proportionally to edge weight, excluding files already in active chat[9]. Fifth, it performs a binary search (lower_bound <= upper_bound) over the ranked definitions until the scope-aware grep_ast TreeContext tree fits within 15% tolerance of max_map_tokens[9].
For file modification, Aider implements 13 Coder strategy subclasses (aider/coders/) after benchmarking showed that matching edit syntax to each model family prevents "lazy coding" (eliding code with comments)[2]. Switching GPT-4 Turbo from SEARCH/REPLACE (diff) to simplified unified diffs (udiff, which omits line numbers in @@ ... @@ hunk headers) raised its laziness benchmark score from **20% to 61%** and reduced editing errors by 9x[31]. Key formats include EditBlockCoder (diff), EditBlockFencedCoder (diff-fenced, placing the file path inside the fence for Gemini models), UnifiedDiffCoder (udiff), WholeFileCoder (whole), PatchCoder (patch), and **ArchitectCoder**, which separates high-level reasoning (an Architect model proposing changes in plain text) from syntactic formatting (an Editor model applying editor-diff or editor-whole)[38]. When EditBlockCoder.do_replace() executes, it attempts an ordered fallback chain: perfect_replace() → replace_part_with_missing_leading_whitespace() (restoring omitted common indentation) → retry without a stray blank first line → try_dotdotdots() (expanding ... elisions if unique) → cross-file fallback across abs_fnames[31]. Note that while a Levenshtein fuzzy matcher exists in editblock_coder.py, Aider disabled it via an early return because full-block fuzzy matching without line anchors occasionally allowed models to silently delete omitted lines[31].

2.6 Google Gemini CLI & Antigravity 2.0 (agy), Factory Droid, Sourcegraph Amp, Goose, Plandex, and Mentat

- **Google Gemini CLI & Antigravity 2.0 (agy)**: **Gemini CLI** (@google/gemini-cli, Apache 2.0, TypeScript, 61.4% on Terminal-Bench 2.0) pairs a ReAct loop (AgentProtocol) and 1M-token context window with hierarchical GEMINI.md files, @path multimodal/file injection, /restore checkpoints, and **5 sandbox backends**: macOS Seatbelt, Docker/Podman, Windows icacls Low Mandatory Integrity Level, Linux **gVisor (runsc)**, and experimental LXC/LXD[3]. In Rombaut's taxonomy, Gemini CLI uniquely implements **LLM Summarization + Verification**, running an automated "Probe" turn after summarization to verify that no critical information was lost[2]. **Google Antigravity 2.0 & Antigravity CLI (agy)** unify Google's agentic IDE and terminal surfaces around a shared harness (42% on the AA Coding Agent Index)[3]. Antigravity introduces **3-layer progressive skill disclosure**—*Layer 1 (Discovery)* reads only YAML metadata at startup; *Layer 2 (Activation)* loads full SKILL.md when matched; *Layer 3 (Execution)* accesses bundled scripts and resources on demand[43]. Its /goal command dynamically defines and

orchestrates parallel subagents (such as running data_analyzer and data_visualizer concurrently once data_cleaner finishes) inside persistent Linux environments and produces reviewable **Verification Artifacts** (Task Lists, Implementation Plans, Walkthroughs, and UI screenshots) to bridge the developer trust gap[42].

- **Factory Droid CLI (droid)**: Built on a Bun SEA runtime and ranking #9 on Terminal-Bench 2.0 (**77.3%**)[3], Factory Droid pioneers **Complexity-to-Model Routing** and **Mission Mode**[47]. Using droid exec --use-spec --spec-model claude-haiku-4-5-20251001, teams route token-heavy codebase reading and specification drafting to a fast model at 0.4x cost before executing changes on Sonnet or Opus across four autonomy tiers (Read-Only, --auto low, --auto medium, --auto high)[48]. **Custom Droids** are Markdown subagents (with built-in worker and explorer droids and direct import from .claude/agents/) that resolve models via a 4-tier hierarchy: (1) pinned frontmatter model, (2) complexity routing (Light, Medium, Heavy), (3) parent inheritance, and (4) org-policy fallback, managed asynchronously via TaskOutput and TaskStop[47]. In **Mission Mode** (droid exec --mission --worker-model <sonnet> --validator-model <opus>), worker models execute parallel tasks in isolated git worktrees (--worktree), while a frontier validator model verifies milestones[48].
- **Sourcegraph Amp & The oracle Subagent**: Amp combines a fast primary agent running locally or in remote cloud **Orbs** (persistent cloud dev machines with OIDC production log access, self-waking schedules, and live web preview portals) with a dedicated read-only subagent tool named **oracle**[45]. Powered by a heavy reasoning model (originally OpenAI o3, then GPT-5 and GPT-6 Astra), oracle is invoked explicitly for complex refactoring specs, cross-service impact analysis, and deep debugging while keeping day-to-day coding fast and economical[45].
- **Block / AAIF Goose, Plandex, and Mentat**: **Goose** (aaif-goose/goose, 55k+ stars, Rust, Apache 2.0, 54.3% Terminal-Bench 2.0) connects 15+ LLM providers to 70+ MCP extensions (including interactive UI **MCP Apps**), runs natively as an **ACP Server** (goose acp), and captures repeatable workflows as parameterized YAML **Recipes** (goose run) and sub_recipes with sequential_when_repeated: true and JSON schema output enforcement for CI/CD[3]. **Plandex** (plandex-ai/plandex, 15.8k stars, MIT) uses a Go server + LiteLLM proxy + PostgreSQL backend to manage 2M-token contexts via tree-sitter project maps and stages edits across 50+ files in a **sandboxed plan version control system** supporting branching, rewinding, and cumulative diff review before applying changes to disk[49]. **Mentat** coordinates multi-file edits using a supervised **Ractor actor model** over a reactive **CloudEvents** bus[50].

Round 3: IDE-Native Extensions, Agentic Editors & Interoperability Protocols (docs/03-ide-and-protocol-agents/)

3.1 The Cline → Roo Code → Zoo Code & Kilo Code Lineage

The open-source IDE agent ecosystem evolved through a direct fork lineage: **Cline** (Apache 2.0) was forked to create **Roo Code**, which grew to 3 million installs before RooCodeInc archived the repository on May 15, 2026 to focus on Roomote[54]. Core Roo contributors

immediately established **Zoo Code (Zoo-Code-Org/Zoo-Code, v3.90.0+)** as the active community continuation, while **Kilo Code** forked the stack to add JetBrains, CLI, Desktop beta, and a 500+ model zero-markup gateway[53].

In Rombaut's source-code taxonomy, **Cline** stands out for two architectural choices: its main execution loop is implemented via **recursion** (maintaining state in recursive stack frames), and it exposes an **LLM-initiated context compression tool (condense)** alongside threshold-triggered compaction[2]. When Cline's internal evaluations showed that LLMs frequently output SEARCH/REPLACE blocks out of file order, Cline built an **order-invariant multi-diff apply algorithm** paired with model-specific markers (---/+++ for Anthropic models and >>>/<<< for Gemini and xAI models), lifting diff edit success rates on Claude Sonnet by nearly **25%**[78]. In its ClineCore SDK runtime, Cline dynamically routes OpenAI/Codex models to apply_patch and other models to editor, never offering both in the same session[87].

Roo Code and its successor **Zoo Code** contribute four major architectural subsystems:

- 🪃 **Orchestrator Mode (Boomerang Tasks)**: Built alongside Code, Architect, Ask, Debug, and custom modes, the **Orchestrator** mode is given restricted default capabilities (no direct file-writing tools) so it stays focused on workflow decomposition[53]. When it delegates work via new_task(mode, message), the parent task pauses and a child subtask launches in an isolated context window under the target specialist mode[55]. When the child finishes and invokes attempt_completion(result), **only the result summary returns ("boomerangs") to the parent task**, preventing intermediate exploration tokens from poisoning the parent's context[55].
- **apply_diff with :start_line: Middle-Out Levenshtein Fuzzy Matching**: Each block in apply_diff requires a :start_line: hint (<<<<<<< SEARCH\n:start_line:10\n-------\n[old]\n=======\n[new]\n>>>>>>> REPLACE)[11]. Rather than scanning the whole file blindly, apply_diff opens a configurable context window (BUFFER_LINES = 40) around :start_line: and executes a **middle-out search using normalized Levenshtein distance** against a 0.8–1.0 confidence threshold[11]. It also tracks consecutiveMistakeCountForApplyDiff per file, stopping after 3 consecutive mistakes to prevent runaway retry billing when a model hits output token limits[11].
- **Shadow Git Checkpoints**: Enabled by default without requiring a user Git repository, the Checkpoint Service initializes an independent **shadow Git repository** that asynchronously commits workspace state at task start and prior to every file modification (excluding node_modules/, dist/, .env, large binaries, and .gitignore paths), enabling instant visual diff comparisons and non-destructive rollbacks[56].
- **Zoo Code (v3.90.0+) Upgrades**: Zoo Code adds **Semble Codebase Intelligence** (automatic, on-demand semantic code search without manual vector DB setup), a **Destructive Command Guard (DCG)** that blocks dangerous shell commands during unattended runs, **duplicate tool-call loop interruption** (warning and halting when an agent repeats the exact same tool call), guarded stream recovery, and per-mode MCP restrictions[53].

3.2 AWS Kiro, Cursor, Devin Desktop (Windsurf), and Continue

- **AWS Kiro (kiro.dev, IDE & Kiro CLI v3)**: Kiro makes **Executable Specs** the primary unit of engineering work[61]. From a feature prompt, Kiro generates three version-controlled Markdown files in sequence before writing code: requirements.md (user stories and acceptance criteria checked via automated reasoning for contradictions and gaps), design.md (architecture, sequence diagrams, and data flow), and tasks.md (an ordered implementation checklist)[60]. Kiro pairs these with **Steering Files** (~/.kiro/steering/ and .kiro/steering/*.md), **Agent Hooks** (.kiro/hooks/ event-driven automations triggered on fileEdit, fileCreate, fileDelete, or userTriggered glob events that automatically update unit tests or API docs on save), and **Property-Based Testing** to verify behavioral invariants across input distributions[60].
- **Cursor, Devin Desktop (Windsurf), and Continue**: **Cursor** pairs low-latency inline completion with Composer/Agent multi-file workflows, a dedicated fast apply model, and a default-on **Auto-review shell sandbox** (macOS Seatbelt, Linux Landlock, WSL2)[13]. **Devin Desktop (formerly Windsurf)** features **Cascade** (automatic context discovery across large enterprise monorepos) and **Flow** (continuous cross-project state)[59]. **Continue (continue.dev)** provides open-source Chat, Plan (read-only), and Agent modes with customizable PR review rules[58].

3.3 Interoperability Protocols: ACP vs. MCP and rulesync

In 2025–2026, the industry standardized on two complementary JSON-RPC 2.0 protocols that decouple editors, agents, and tools[65]:

Architectural Dimension	Agent Client Protocol (ACP)	Model Context Protocol (MCP)
Origin & Governance	Created by Zed Industries (Aug 2025), co-developed with JetBrains & Google; Apache 2.0[65]	Created by Anthropic; open industry standard[52]
Layer & Problem Solved	**Outer Layer (Editor ↔ Agent)**: "LSP for AI Coding Agents"—lets any IDE drive any coding agent (N + M instead of N × M glue code)[65]	**Inner Layer (Agent ↔ Tools/Data)**: Connects an AI agent to external APIs, databases, browsers, and resources[65]
Client vs. Server Role Inversion	**Code Editor = Client** (owns UI, diff review, permission dialogs); **Coding Agent = Subprocess / Server**[65]	**Coding Agent = Host / Client**; **Tool Integration = Subprocess / Server**[66]

Transport & Core Methods	JSON-RPC 2.0 over stdio (local subprocess) or HTTP/WebSocket (remote); session/initialize, session/new, session/prompt, elicitation, cancellation[65]	JSON-RPC 2.0 over stdio, SSE, or Streamable HTTP; tool discovery, tools/call, resources, prompts[35]
Adopting Ecosystem (Oct 2026)	**Editors**: Zed, JetBrains IDEs, Neovim (agentic.nvim), Emacs, VS Code. **Agents**: Gemini CLI (--acp), Claude Code (claude-agent-acp), Codex (codex-acp), Goose (goose acp), Cline, OpenHands, DeepSeek Harness (dsh-acp), Kimi CLI[65]	Supported by 62 of 65 tracked coding agents in Terminal Trove (excluding Aider, Pi core by design, and Nanocode)[3]

To bridge configuration formats across tools, **dyoshikawa/rulesync** provides a unified CLI (rulesync generate --targets "*" --features "*" and rulesync convert --from cursor --to copilot,claudecode) that compiles .rulesync/ into target-specific rules (AGENTS.md, CLAUDE.md, .cursorrules, GEMINI.md, zoocode), commands, MCP configs, ignore files, subagents, and skills[63]. Complementing this, **freeCodeCamp/agent-scaffold** creates new agent-ready projects with AGENTS.md (and CLAUDE.md symlinked to it), .agents/skills/, and Docker/Devcontainer files[62], while **soumatheusgomes/vibe-coding-toolkit** supplies production quality gates (350-line file limits via ESLint/Biome, RTK token proxy to compress repetitive terminal output, Subagent Waves, and bounded Harness² trial workflows)[64].

Round 4: Autonomous SWE Research Scaffolds & Taxonomic Synthesis (docs/04-swe-research-scaffolds/)

4.1 Princeton/Stanford's SWE-agent (ACI & SWE-ReX) vs. mini-SWE-agent v2

The Princeton NLP and Stanford team created both **SWE-agent** (NeurIPS 2024) and its successor **mini-SWE-agent v2** (Feb 2026), illustrating how scaffold requirements shift as base models improve[7]:

SWE-agent demonstrated that giving GPT-4 Turbo raw Linux shell commands (cat, sed, unbounded grep -rn) degraded SWE-bench Lite resolution by 10.7 percentage points because unbounded outputs overflow context and unvalidated edits introduce syntax errors[7].

SWE-agent introduced an **Agent-Computer Interface (ACI)** implemented as YAML+bash tool bundles (tools/<bundle>/bin/) inside a **SWE-ReX** sandbox[67]. Its file viewer (open, goto, scroll_up, scroll_down) displays bounded 100-line windows (WINDOW=100) with a 2-line overlap and persists cursor state (CURRENT_FILE, FIRST_LINE) in environment files[7]. Its search commands

(search_dir, search_file, find_file) cap results at 50–100 files and explicitly return "Your command ran successfully and did not produce any output" on empty stdout[7]. Crucially, its line-range editor (edit <start>:<end>\n<replacement>\nend_of_edit) validates bash actions with bash -n before execution and runs a linter (flake8) immediately after every file edit, **automatically reverting the edit** if syntax errors are introduced[67]. To run stateful tools (gdb, ipython) across Local, Docker, Modal, and AWS Fargate backends without fragile timeouts, **SWE-ReX** appends a unique completion sentinel (; echo "###SWE-REX-COMPLETE-<uuid>###") to every command in a persistent shell session[67].
By 2025–2026, frontier models became strong enough at direct shell usage that the authors placed SWE-agent into maintenance mode and released **mini-SWE-agent v2** (~100 lines of core Python)[69]. mini-swe-agent eliminates tool-calling schemas entirely—using only bash blocks in triple backticks—and executes every action via a **stateless subprocess.run call** rather than a persistent shell session[68]. Because no shell state carries over between turns, failed steps never poison subsequent commands, and sandboxing reduces to swapping subprocess.run for docker exec, Podman, Singularity, Bubblewrap, or Nebius Contree[68]. Its message history is strictly linear (trajectory === messages), and control signals pass via InterruptAgentFlow exceptions (Submitted, LimitsExceeded, FormatError in default.py:88-96)[2]. On the official SWE-bench Verified leaderboard, mini-SWE-agent v2.0.0 resolves **76.80%** with Claude 4.5 Opus ($0.75/task average) and **75.80%** with MiniMax M2.5 ($0.07/task average)[12].

4.2 OpenHands, Agentless, AutoCodeRover / SpecRover, Moatless / SWE-Search, Prometheus, DARS, and SWE-AF

- **OpenHands (OpenHands/OpenHands, v1 SDK & Agent Canvas)**: Built on an **event-sourced architecture** where all agent actions and environment responses flow through an immutable EventStream (memory/view.py:13-96 computes compressed context views dynamically)[2]. Its generalist CodeActAgent combines stateful bash (pexpect), file editing, and **BrowserGym** web interaction inside Docker or APIRemoteWorkspace cloud sandboxes, while **Agent Canvas** (@openhands/agent-canvas) and the Automation Server coordinate parallel runs triggered by Slack, Linear, Jira, and GitHub CI webhooks[71].
- **Agentless (OpenAutoCoder/Agentless, ~700 lines)**: Demonstrated that a deterministic 3-phase pipeline without autonomous tool selection can achieve competitive bug resolution at low cost ($0.34–0.43/issue; 51.8% on SWE-bench Verified with Kimi K2 Instruct)[93]. Phase 1 (**Hierarchical Localization**) narrows candidates from *File-level* (repo tree + text-embedding-3-small similarity) to *Element-level* (condensed AST class/function skeleton signatures) to *Line-level* (4 sampled edit location sets)[73]. Phase 2 (**Repair**) builds ±10-line context windows and samples 40 SEARCH/REPLACE diffs (4 location sets × 10 patches)[73]. Phase 3 (**Patch Validation**) synthesizes a standalone reproduction test, filters the 40 candidates against regression and reproduction tests, normalizes surviving patches via **AST parsing and unparsing**, and selects the majority-vote winner[73].
- **AutoCodeRover & SpecRover (zanwenfu/auto-code-rover ⟶ Sonar Foundation**

Agent, #1 on SWE-bench Verified at 79.20%): Treats the repository as an Abstract Syntax Tree (AST) rather than plain text files[12]. After GenerateReproducer writes a bug-reproducing test, GenerateSBFL runs **Spectrum-Based Fault Localization (SBFL)**—comparing control-flow coverage of passing vs. failing tests to rank suspicious methods[74]. SearchAgent then conducts a stratified search across 8 languages using tree-sitter AST APIs (search_class, search_method_in_class, search_code_in_file, get_method_signature) and extracts intended function specifications (**SpecRover**)[75]. Orchestrated as a Python generator coroutine over immutable ActionState snapshots (recording/*.json), if WritePatch or ReviewPatch fails, the **SelfFix Agent** (agent_self_fix.py) diagnoses which upstream stage caused the failure (e.g., ContextRetrieval proposing the wrong class), generates corrective feedback for that stage, and **replays the pipeline from that exact snapshot**[75].

- **Moatless Tools & SWE-Search (aorwall/moatless-tools, arXiv:2410.20285, 70.80% SWE-bench Verified)**: Replaces linear retries with **Monte Carlo Tree Search (MCTS)** over file-context state snapshots, yielding a 23% relative improvement across five evaluated models[12]. Node selection uses a modified depth-aware Upper Confidence Bound for Trees (UCT) criterion that encourages broad shallow exploration ($\alpha e^{-\beta(d-1)}$) and deeper convergence ($-\gamma\sqrt{d}$)[77]:

$$UCT(s,a) = V(s,a) + C_s\sqrt{\frac{\ln N(s)}{N(s,a)}} + \alpha e^{-\beta(d-1)} - \gamma\sqrt{d}$$

  At each node, the **Action Agent** executes edits and tests, the **Value Agent** outputs both a numerical reward estimate v_t and a qualitative natural-language explanation ϵ_t ("hindsight feedback" passed back to parent nodes so subsequent expansions avoid prior mistakes), and a multi-agent **Discriminator Agent** debates up to 5 final candidate patches before a Judge selects the submission[77].
- **Prometheus, DARS-Agent, and SWE-AF**: **Prometheus** (74.40% SWE-bench Verified) uses nested LangGraph subgraphs where each node maintains an isolated message list (edit_messages, analyzer_messages) reset on retry and binds stage-restricted tools[2]. **DARS-Agent** builds a greedy trajectory tree (dars_agent.py:294-297) with an LLM judge and Docker action replay[2]. **SWE-AF** (Agent-Field/SWE-AF) compiles PRDs into an **Issue DAG** driven by three nested loops (Outer Replanner, Middle Issue Advisor with FAILED_NEEDS_SPLIT sub-issue injection, Inner Coder/Verifier), tracks incompleteness as a typed data structure, and signs every inter-agent call with a cryptographic **Verifiable Credential (VC)**[76].

Source-Code Taxonomy Matrix: How 16 Frontier Coding Scaffolds Implement Control, Tools, and State

Grounded in Rombaut's (2026) 12 source-code dimensions across 3 taxonomic layers + **OmniHarness** synthesis

Click any agent row to inspect source files & 12-dimension details

All Layers (12 Dims) | Layer 1: Control | Layer 2: Tools & Sandbox | Layer 3: State & Compaction

Filter agent, file, or patter | Expand Code Paths

Layer 1 (Control): Recursive / Tree / Delegation **Layer 2 (Tools/Edit):** Dynamic MCP / Fuzzy Diff **Kernel/Container Sandbox:** Seatbelt / bwrap / Docker

Layer 3 (State): DAG / SQLite / EventStream **Compaction & Routing:** Probe / Rehydration / Multi-Model **Hazard:** Destructive / Crash

SCAFFOLD / AGENT	LAYER 1: CONTROL ARCHITECTURE (DIMS 1–4)		LAYER 2: TOOL & ENVIRONMENT INTERFACES (DIMS 5–8)			LAYER 3: RESOURCE & STATE MANAGEMENT (DIMS 9–12)		
AGENT & ENTRYPOINT Source / License	DIMS 1, 2, 4 Control Loop & Recovery	DIM 3 Sub-Agent Delegation	DIMS 5, 6 Tool Surface & Discovery	DIM 7 Code Edit Protocol	DIM 8 Sandbox Isolation	DIM 9 State Data Structure	DIM 10 Context Compaction	DIMS 11, 12 Model Routing & Budget
Harness² Apache-2.0 harness2/ (--substrate…	2-Level PPC Propose-Probe…	Subagents + Plugins 8 Editable Co…	Progressive Skills Scripts + Too…	Substrate-Delegated OpenCode / Co…	Host / Substrate Codex sandbox…	Improvement Harness Contrastive T…	Experience Reflector Cross-task le…	Fixed-Weight Opt Parallel K / …
DeepSeek … OSS (dsh) core/agent-loop, ctx.t…	Cordis Waterfall turn/* → ste…	Scoped Realm Fork ctx.agents.cr…	Scoped Registry ctx.tools wat…	Capability Seams fs/* & tools …	Profile Bundles web/headless/…	SessionEvent Log Append-only c…	Projection Seam deriveMessage…	Adapter Seam ctx.llm model…
Claude … Proprietary claude -p (Auto Classi…	While Loop Auto Classifi…	Subagents + Cache Fork Task sub-agen…	Deferred Search Dynamic MCP +…	Exact String Deterministic…	Opt-in Seatbelt+bwrap Shell-only is…	JSONL DAG Session trans…	3-Tier + Rehydration Auto context …	Role-Based (Claude) Opus 5.5 / So…
Codex CLI Apache-2.0 codex.rs:6156–6164	While Loop codex exec + …	ThreadManager Sub-agent gen…	20+ Rebuilt/Round built_tools()…	apply_patch Free-mode pat…	Default Seatbelt+bwrap On by default…	Flat List (Filtered) Raw history p…	Pre/Mid-Turn Summary Scaffold-trig…	Multi-Model + Local GPT-6.1 Sol /…
OpenCode MIT message-v2.ts, agent.t…	While Loop opencode run	Role Subagents build/plan/ex…	18+ Dynamic + MCP Core + Plugin…	4-Pass Patch edit function…	Host (None) Denies .env r…	Typed SQLite Log 12 part types…	2-Stage Prune+Summary Prunes old ou…	Role-Based (75+) Per-agent mod…
Pi (pi-mono) MIT @earendil-works/pi-age…	While Loop (Stream) pi -p / Diffe…	Extension Hooks /fork, /clone…	4 Core + Skills read, write, …	Exact String TypeBox schem…	Host (No Prompts) Unrestricted …	JSONL DAG 8-char id & p…	Reserve + Branch Sum 20k kept / 16…	Cross-Provider Handoff 4 wire protoc…
Aider Apache-2.0 base_coder.py:1024–10…	Gen-Test-Repair Architect → …	Architect / Editor architect_cod…	Static (RepoMap) Fixed at init…	13 Coder Formats Model-specifi…	Host (Subprocess) Local git & l…	Destructive Overwrites do…	Hierarchical Summary Recursive LLM…	Role-Based (3-Model) models.py:687…
Roo / Zoo … Apache-2.0 apply_diff (Boomerang …	While Loop Mode-driven R…	Boomerang Modes Parent/child …	Dynamic MCP Mode-scoped t…	Levenshtein apply_diff Fuzzy search/…	Host (IDE Workspace) Command allow…	Flat List Per-task JSON…	4-Layer Delta Sliding windo…	Mode-Based Per-mode LLM …
Cline Apache-2.0 api/index.ts:76–149	Plan-Execute Loop Dual Plan / A…	None (Single Loop) In-loop MCP t…	27+ Flat + Dynamic MCP Largest flat …	Write + Search/Replace Full write + …	Host (User Approval) Per-step IDE …	Flat List Task conversa…	LLM condense Tool LLM-initiated…	Mode-Based (Plan/Act) api/index.ts:…
G… Apache-2.0 / Prop. tool-registry.ts:312–…	While Loop gemini -p / a…	Subprocess Discovery Tool discover…	17+ Dynamic + MCP tool-registry…	Write + Search/Replace Function call…	5-6 Backends Seatbelt/Dock…	Flat List (Filtered) Raw history p…	Probe-Verified Summary Summary + ver…	Gemini / Multi-Tier Gemini 4 Argo…
Factory … Proprietary droid exec (Autonomy P…	While Loop droid exec he…	Mission Worktrees Parallel git …	Dynamic MCP + Policy Air-gapped & …	Exact String Policy-checke…	Opt-in Seatbelt+bwrap + Network fil…	Flat List Session state…	3-Tier + Rehydration Artifact-pres…	Multi-Window Fallback 5 Provider fa…
Sourceg… Proprietary amp -x (Mode + Oracle)	While Loop amp -x zero-p…	Subagents + Oracle Reasoning Ora…	Dynamic MCP Executes with…	Exact String Fast determin…	Host / Cloud Machines No local OS s…	Flat List Synced web/CL…	Scaffold Summary Threshold thr…	Complexity Tier + Oracle Picked per ex…
SWE-agent MIT agent_configs, str_rep…	While Loop (ReAct) Retry loop wi…	None Single-agent …	3–35 Config-Scoped 15 toolkits (…	str_replace (10 Parsers) XML / Thought…	Docker SWE-ReX Containerized…	Flat List (Filtered) Raw history p…	Head-Tail Rule 7 processors …	Per-Attempt Rotation Rotates on re…
mini-SWE-agent MIT default.py:88–96	While Loop (Minimal) default.py:88…	None Zero sub-agen…	1 Tool (Static Bash) All actions v…	Bash Only sed / python …	Docker / Subprocess Containerized…	Flat List (Unfiltered) Messages sent…	None (Crash) ContextWindow…	Single Single fixed …
OpenHands MIT memory/view.py:13–96	Event-Driven Loop Action-Observ…	Agent Delegation Sub-agent dis…	Config-Conditioned Dynamic toolk…	str_replace_editor Function call…	Docker Runtime Isolated cont…	EventStream memory/view.p…	Scaffold LLM Summary Compression m…	MultimodalRouter llm/router/ru…
AutoCode… GPL / MIT agent_proxy.py:81, Act…	Fixed Pipeline / MCTS HardcodedMeta…	Stage Agents Search/Patch/…	Stage-Scoped AST 8 AST APIs / …	XML / Pydantic AST lint & di…	Docker / Testbed SBFL & reprod…	Frozen ActionState / MCTS recording/*.j…	Structured Isolation 15-turn limit…	Per-Attempt Rotation Majority vote…
OmniHarn… Synthesis Best-of-Breed Unified A…	2-Level PPC + Waterfall Cordis hooks …	Cache Fork + Worktrees Boomerang + S…	4 Core + Deferred AST Progressive S…	4-Pass + Levenshtein apply_patch +…	Default Seatbelt+bwrap + Docker SWE-…	JSONL DAG + SQLite Log EventStream +…	3-Tier + Probe-Verified Rehydration +…	Complexity Tier + Oracle Cross-provider

Architectural classification of 16 primary coding agents and harnesses across Layer 1 (Control Loop & Delegation), Layer 2 (Tool Surface, Edit Protocol, & Sandbox Isolation), and Layer 3 (State Structure, Context Compaction, & Model Routing).

Data sources: Inside the Scaffold: A Source-Code Taxonomy of Coding Agent Architectures (arXiv:2604.03515) · Google Research (Harness²) · DeepSeek AI (DeepSeek Harness Architecture) · Developers Digest (Pi Agent Toolkit Architecture) · AutoCodeRover & SpecRover · Dbggr (AI Coding Agents Compared, Oct 2026)

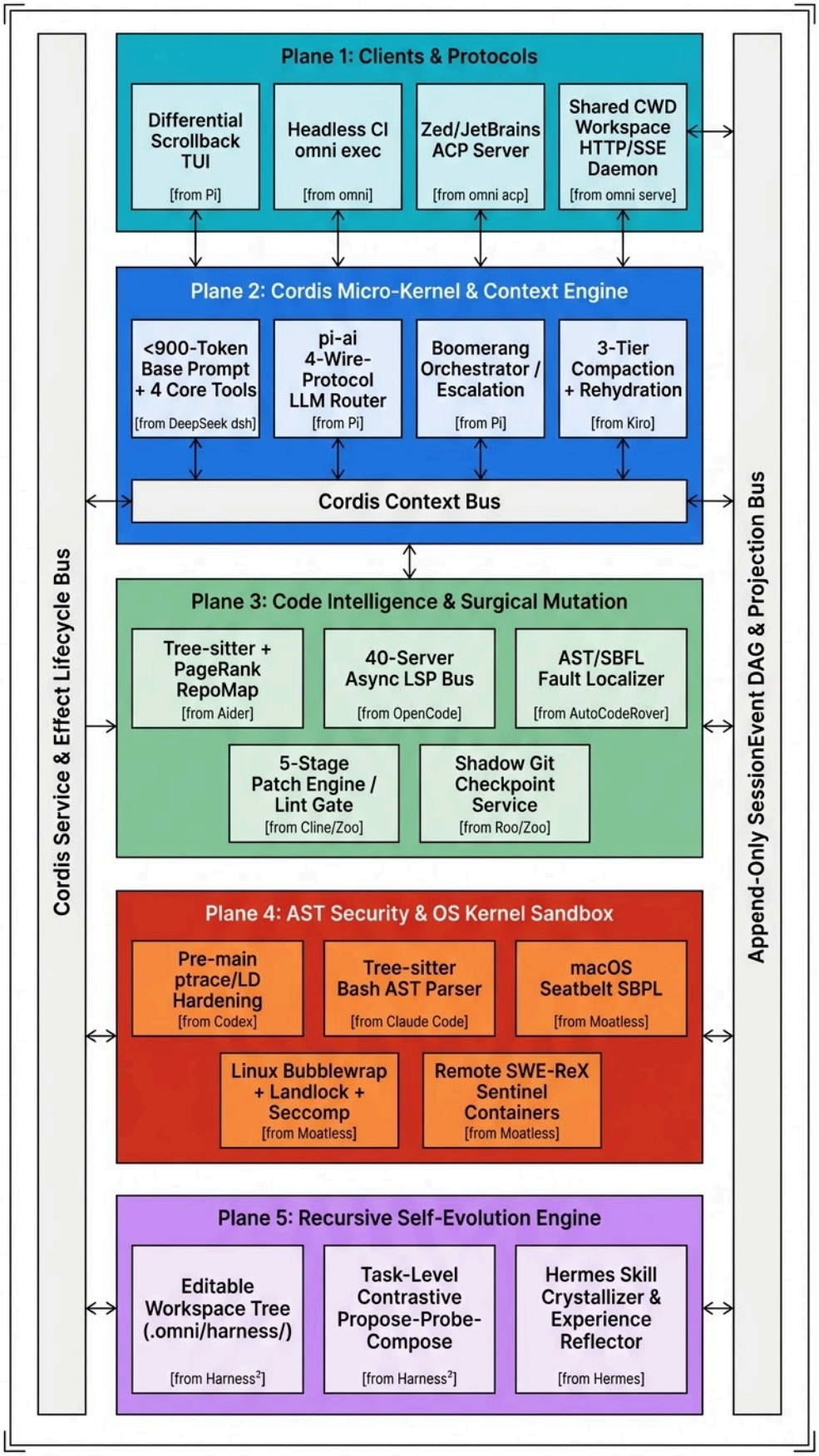
Part 3: OmniHarness (omni) — One CLI and Agent Scaffold to Rule Them All

Synthesizing the empirical findings and source-code teardowns from Rounds 1–4, **OmniHarness (omni)** is designed around four engineering invariants: **a sub-900-token micro-kernel at Turn 1, Cordis spatiotemporal plugin extensibility at runtime, deterministic AST/LSP verification on every file mutation, and contrastive Harness² self-evolution across tasks.**

3.1 Resolving the Five Core Architectural Paradoxes

Architectural Paradox	Failure Mode in Existing Agents	OmniHarness Unified Resolution & Pedigree
1. The "Harness Tax" Paradox (Capability vs. Turn-1 Token Cost)	Heavyweight harnesses send 7,025–16,581 tokens on Turn 1 and MCP servers dump 13,700+ schema tokens, multiplying API spend 3x–5x without improving baseline solve rates[5].	**Pi's <900-token micro-kernel** exposing only **4 core tools** (read, edit, write, bash) + a 1-line index of skills/MCPs at Turn 1, combined with **Codex's deferred tool search** and **Antigravity's 3-layer progressive skill disclosure**[4].
2. The Extensibility Paradox (Minimalist Loop vs. Deep Customization)	Minimalist loops (mini-swe-agent, pi) lack built-in enterprise sandboxing or LSP buses, while monolithic agents (Claude Code) lock the loop inside closed binaries[16].	Mounts the micro-kernel onto **DeepSeek Harness's Cordis Engine** (ctx.llm, ctx.tools, ctx.sessions, ctx.sandbox, ctx.repomap, ctx.lsp, ctx.evolver) with inject spatial dependencies, ctx.effect() reversible teardown, and **Creator Mode** live plugin synthesis[16].
3. The Code Edit Paradox (Strict Exact Match vs. Fuzzy Corruption)	Exact string match (Claude Code) fails on minor indentation drift; unanchored fuzzy matching silently deletes lines or edits	**5-Stage Surgical Patch Engine**: **Cline's order-invariant multi-diff** + **Codex's 4-pass normalizer** + **Roo/Zoo's**

	the wrong duplicate block[31].	**:start_line: middle-out Levenshtein (BUFFER_LINES=40)** + **SWE-agent's lint-on-edit auto-rollback** + **OpenCode's async LSP textDocument/didChange diagnostics**[8].
4. The Long-Horizon State Paradox (Context Overflow vs. Compaction Amnesia)	Unbounded logs crash (mini-swe-agent) or halt on output caps (OpenCode), while repeated LLM summaries lose early architectural decisions (Claude Code after 60+ compactions)[2].	**Append-Only JSONL Event DAG** (*Pi* + *DeepSeek*) + **Shadow Git Checkpoints** (*Roo/Zoo*) + **3-Tier Compaction**: disk spill microcompaction (*DeepAgents/Claude*) + **Probe-Verified Summarization** (*Gemini CLI*) + **6-Step Rehydration** restoring top 5 files, Kiro Specs, and SWE-AF Issue DAG[2].
5. The Static Playbook Paradox (Frozen Prompts vs. Open-World Tasks)	Static prompts and fixed tool rules degrade when encountering unfamiliar repositories, domain conventions, or novel toolchains[10].	Built-in omni evolve running **Google Harness²'s Propose–Probe–Compose loop** over the workspace .omni/harness/c1..c8 tree with 3 fail-closed validation gates, paired with **Nous Hermes Agent's Skill Crystallizer**[14].


Plane 1: Clients & Protocols
Differential Scrollback TUI
[from Pi]
Headless CI omni exec
[from omni]
Zed/JetBrains ACP Server
[from omni acp]
Shared CWD Workspace HTTP/SSE Daemon
[from omni serve]
Plane 2: Cordis Micro-Kernel & Context Engine
<900-Token Base Prompt + 4 Core Tools
[from DeepSeek dsh]
pi-ai 4-Wire-Protocol LLM Router
[from Pi]
Boomerang Orchestrator / Escalation
[from Pi]
3-Tier Compaction + Rehydration
[from Kiro]
Cordis Context Bus
Plane 3: Code Intelligence & Surgical Mutation
Tree-sitter + PageRank RepoMap
[from Aider]
40-Server Async LSP Bus
[from OpenCode]
AST/SBFL Fault Localizer
[from AutoCodeRover]
5-Stage Patch Engine / Lint Gate
[from Cline/Zoo]
Shadow Git Checkpoint Service
[from Roo/Zoo]
Plane 4: AST Security & OS Kernel Sandbox
Pre-main ptrace/LD Hardening
[from Codex]
Tree-sitter Bash AST Parser
[from Claude Code]
macOS Seatbelt SBPL
[from Moatless]
Linux Bubblewrap + Landlock + Seccomp
[from Moatless]
Remote SWE-ReX Sentinel Containers
[from Moatless]
Plane 5: Recursive Self-Evolution Engine
Editable Workspace Tree (.omni/harness/)
[from Harness²]
Task-Level Contrastive Propose-Probe-Compose
[from Harness²]
Hermes Skill Crystallizer & Experience Reflector
[from Hermes]
Cordis Service & Effect Lifecycle Bus
Append-Only SessionEvent DAG & Projection Bus

3.2 Workspace Layout & c1–c8 Editable Harness Tree (.omni/)

When omni init runs in a repository (combining freeCodeCamp/agent-scaffold and dyoshikawa/rulesync conventions with Google Research's Harness² c1–c8 structure[14]), it creates a unified .omni/ directory and symlinks AGENTS.md, CLAUDE.md, and GEMINI.md to .omni/harness/c1-system-prompt.md so all coding tools share a single source of truth:

Component ID	Workspace Path	Runtime Role & Provenance
Root Config	.omni/cordis.yml & .omni/cordis.patch.yml	5-layer declarative plugin composition, model routing tiers, and sandbox policy[19]
c1 System Prompt	.omni/harness/c1-system-prompt.md (symlinked to AGENTS.md, CLAUDE.md)	Sub-900-token core operating rules and build/test commands (Pi + Harness² c1)[4]
c2 Guardrails	.omni/harness/c2-guardrails.yml	Destructive Command Guard (DCG) rules, .git/.omni write locks, and leak_globs (Zoo Code + Harness² c2)[15]
c3 Memory & Specs	.omni/harness/c3-memory/ (lessons.jsonl, specs/{requirements,design,tasks}.md, issue-dag.json)	Persistent Harness² reflection ledger, Kiro executable specs, and SWE-AF Issue DAG[15]
c4 Tool Guidance	.omni/harness/c4-tool-guidance.md	Concise heuristics for when to invoke oracle, subagent, or deferred MCP search (Harness² c4)[15]
c5 Subagents	.omni/harness/c5-subagents/*.md (explorer.md, worker.md, oracle.md, guardian.md)	Reusable Markdown subagents with YAML frontmatter for complexity-to-model routing (Claude Code + Factory Droid + Harness² c5)[15]

c6 Skills	.omni/harness/c6-skills/<skill-name>/SKILL.md	Progressive-disclosure domain & methodology playbooks; only YAML name/description loaded at Turn 1 (Pi + Antigravity + Hermes + Harness² c6)[4]
c7 Scripts	.omni/harness/c7-scripts/<skill-name>/bin/*	Lint-validated YAML+bash ACI scripts and PTC programmatic tool routines (SWE-agent + DeepSeek PTC + Harness² c7)[15]
c8 Plugins & Hooks	.omni/harness/c8-plugins/*.ts & .omni/harness/c8-hooks/*.yaml	jiti hot-reloaded Cordis plugins and Kiro-style file-event hooks (onFileSave, PreToolUse, SubagentStop)[4]
Session & Spill Store	.omni/sessions/<uuid>.jsonl, .omni/state.sqlite, .omni/spill/, .omni/shadow-git/	Append-only JSONL DAG log, SQLite FTS5 index, cold-storage tool output spills, and Shadow Git checkpoint repo[20]

3.3 Core Implementation Blueprint of OmniHarness

Below are the five core architectural modules that implement OmniHarness.

Module 1: The Cordis Spatiotemporal Micro-Kernel, Turn/Step Waterfall, and Append-Only Event DAG (packages/core/src/kernel.ts)

This module fuses **DeepSeek Harness's Cordis service/waterfall architecture** (ctx.sessions, ctx.tools, ctx.agentLoop, deriveMessages()) with **Pi's append-only JSONL DAG** (id, parentId, crash-recovery register) and **automatic mid-step continuation** (fixing OpenCode's 32k output truncation bug identified in the Shandong study)[5].

```TypeScript
// packages/core/src/kernel.ts
import { createHash, randomUUID } from "node:crypto"
import * as fs from "node:fs/promises"
```

```
export type Branded<T, B extends string> = T & { readonly __brand: B };
export type SessionId = Branded<string, "SessionId">;
export type NodeId = Branded<string, "NodeId">;

// Append-Only DAG Entry (Pi + DeepSeek Harness SessionEvent union)
export interface DagSessionEvent {
  readonly id: NodeId;
  readonly parentId: NodeId | null;
  readonly sessionId: SessionId;
  readonly timestamp: number;
  readonly type:
    | "session/header"
    | "turn/start"
    | "step/start"
    | "user/message"
    | "assistant/message"
    | "tool/call"
    | "tool/result"
    | "compaction/boundary"
    | "branch/summary"
    | "checkpoint/shadow_commit"
    | "model/change"
    | "step/end"
    | "turn/end";
  readonly payload: Record<string, unknown>;
  readonly ignorable?: boolean;
}

export type Disposer = () => void;
export type WaterfallNext<T> = () => Promise<T>;
export type WaterfallListener<C, T> = (ctx: C, next: WaterfallNext<T>) => Promise<T>;

// Cordis Spatiotemporal Context Kernel (Spatial inject + Temporal ctx.effect disposers)
export class CordisContext {
  private services = new Map<string, unknown>();
  private effects = new Map<string, Disposer[]>();
  private waterfalls = new Map<string, WaterfallListener<any, any>[]>();

  public provide<T>(key: string, service: T, ownerPluginId = "core"): Disposer {
    this.services.set(key, service);
    const dispose = () => { this.services.delete(key); };
    this.effect(dispose, ownerPluginId);
    return dispose;
```

```
  public resolve<T>(key: string): T {
    const svc = this.services.get(key);
    if (!svc) throw new Error(`CORDIS_SERVICE_UNAVAILABLE: Missing required service '${key}'`);
    return svc as T;
  }

  public effect(disposer: Disposer, ownerPluginId = "core"): void {
    const list = this.effects.get(ownerPluginId) ?? [];
    list.push(disposer);
    this.effects.set(ownerPluginId, list);
  }

  public unloadPlugin(pluginId: string): void {
    const disposers = this.effects.get(pluginId) ?? [];
    for (const dispose of disposers.reverse()) dispose();
    this.effects.delete(pluginId);
  }

  public onWaterfall<C, T>(event: string, listener: WaterfallListener<C, T>, ownerPluginId =
"core"): Disposer {
    const chain = this.waterfalls.get(event) ?? [];
    chain.push(listener);
    this.waterfalls.set(event, chain);
    const dispose = () => {
      const idx = chain.indexOf(listener);
      if (idx >= 0) chain.splice(idx, 1);
    };
    this.effect(dispose, ownerPluginId);
    return dispose;
  }

  public async emitWaterfall<C, T>(event: string, payload: C, terminal: () => Promise<T>):
Promise<T> {
    const chain = this.waterfalls.get(event) ?? [];
    let index = -1;
    const dispatch = async (i: number): Promise<T> => {
      if (i <= index) throw new Error(`Waterfall next() called multiple times in ${event}`);
      index = i;
      const fn = chain[i];
      if (!fn) return terminal();
      return fn(payload, () => dispatch(i + 1));
```

```
  }
  return dispatch(0)
}

// Append-Only DAG Session Store with "Model-Visible Means Logged" Projection
export class DagSessionStore {
  private events = new Map<NodeId, DagSessionEvent>()
  private activeLeaf: NodeId | null = null

  constructor(private readonly logFilePath: string, private readonly registerPath: string) {}

  public async append(
    sessionId: SessionId,
    type: DagSessionEvent["type"],
    payload: Record<string, unknown>
  ): Promise<DagSessionEvent> {
    const id = createHash("sha256").update(randomUUID()).digest("hex").slice(0, 8) as NodeId;
    const event: DagSessionEvent = {
      id,
      parentId: this.activeLeaf,
      sessionId,
      timestamp: Date.now(),
      type,
      payload,
    };
    // 1. Write durable event to append-only JSONL DAG
    await fs.appendFile(this.logFilePath, JSON.stringify(event) + "\n", "utf8");
    // 2. Overwrite single crash-recovery program-counter register (Pi architecture)
    await fs.writeFile(this.registerPath, JSON.stringify({ activeLeaf: id, ts: event.timestamp }),
"utf8");
    this.events.set(id, event);
    this.activeLeaf = id;
    return event;
  }

  // Deterministic walk from active leaf to root honoring compaction boundaries
  public deriveMessages(leafId: NodeId | null = this.activeLeaf): Array<{ role: string; content:
unknown }> {
    const branch: DagSessionEvent[] = [];
    let cursor = leafId;
    while (cursor) {
      const ev = this.events.get(cursor);
```

```
    if (!ev) break
    branch.push(ev)
    if (ev.type === "compaction/boundary") break  // Stop walk at latest compaction boundary
    cursor = ev.parentId
  }
  branch.reverse()
  return branch
    .filter((e) => e.type === "user/message" || e.type === "assistant/message" || e.type ===
"tool/result" || e.type === "compaction/boundary")
    .map((e) => ({
      role: e.type === "user/message" ? "user" : e.type === "assistant/message" ? "assistant" : "tool",
      content: e.payload,
    }))
}

// Jump to any historical node in the DAG (/tree or MCTS backtracking)
public checkoutBranch(targetNodeId: NodeId): void {
  if (!this.events.has(targetNodeId)) throw new Error(`Unknown DAG node: ${targetNodeId}`)
  this.activeLeaf = targetNodeId
}
}
```

Module 2: Five-Stage Fault-Tolerant Surgical Patch Engine with Lint-on-Edit Rollback & LSP Diagnostic Injection (packages/tools/src/surgical-edit.ts)
This module combines **Cline's order-invariant multi-block parser**, **OpenAI Codex's 4-pass whitespace/Unicode normalizer**, **Roo/Zoo Code's :start_line: middle-out Levenshtein fuzzy matcher (BUFFER_LINES = 40)**, **SWE-agent's immediate lint-on-edit auto-revert**, **Roo/Zoo's 3-strike circuit breaker**, and **OpenCode's LSP textDocument/didChange diagnostic loop**[8].

```
TypeScript
// packages/tools/src/surgical-edit.ts
import * as fs from "node:fs/promises"

export interface SearchReplaceBlock {
  startLineHint?: number
  search: string
  replace: string
}
```

```ts
export interface LspDiagnostic {
  line: number;
  severity: "error" | "warning";
  message: string;
}

export interface LspClientBus {
  notifyDidChange(filePath: string, newContent: string): Promise<LspDiagnostic[]>;
}

export interface ShadowGitService {
  recordPreEditCheckpoint(filePath: string): Promise<string>;
}

// Levenshtein similarity for Roo/Zoo middle-out fuzzy matching
function levenshteinSimilarity(a: string, b: string): number {
  if (a === b) return 1.0;
  const maxLen = Math.max(a.length, b.length);
  if (maxLen === 0) return 1.0;
  const dp = Array.from({ length: a.length + 1 }, (_, i) => i);
  for (let j = 1; j <= b.length; j++) {
    let prev = dp[0];
    dp[0] = j;
    for (let i = 1; i <= a.length; i++) {
      const temp = dp[i];
      dp[i] = b[j - 1] === a[i - 1] ? prev : 1 + Math.min(prev, dp[i], dp[i - 1]);
      prev = temp;
    }
  }
  return 1.0 - dp[a.length] / maxLen;
}

// Codex seek_sequence.rs 4-pass line normalization
function normalizeLine(line: string, pass: 1 | 2 | 3 | 4): string {
  if (pass === 1) return line;
  if (pass === 2) return line.replace(/\s+$/, "");
  if (pass === 3) return line.trim();
  return line
    .trim()
    .replace(/[\u2018\u2019]/g, "'")
    .replace(/[\u201C\u201D]/g, '"')
    .replace(/[\u2013\u2014]/g, "-");
}
```

```
export class SurgicalPatchEngine {
  private consecutiveMistakes = new Map<string, number>();
  private readFilesInSession = new Set<string>();
  private readonly BUFFER_LINES = 40;
  private readonly FUZZY_THRESHOLD = 0.88;

  public markFileRead(filePath: string): void {
    this.readFilesInSession.add(filePath);
  }

  public async applySurgicalEdits(
    filePath: string,
    blocks: SearchReplaceBlock[],
    lspBus: LspClientBus,
    shadowGit: ShadowGitService
  ): Promise<{ success: boolean; output: string }> {
    // 1. Read-Before-Mutate Guard (Claude Code / Coding-Agent Harness rule)
    if (!this.readFilesInSession.has(filePath)) {
      return {
        success: false,
        output: `REJECTED: You must call 'read' on '${filePath}' before editing it so your context anchors to current disk state.`
      };
    }

    const originalContent = await fs.readFile(filePath, "utf8");
    const fileLines = originalContent.split(/\r?\n/);

    // 2. Locate all blocks first for Order-Invariant Multi-Diff Application (Cline algorithm)
    const resolvedSpans: Array<{ startIdx: number; endIdx: number; replacementLines: string[]; origSearchFirstLine: string }> = [];

    for (const block of blocks) {
      const searchLines = block.search.split(/\r?\n/);
      const replaceLines = block.replace.split(/\r?\n/);
      const match = this.findBlockSpan(fileLines, searchLines, block.startLineHint);

      if (!match) {
        const mistakes = this.consecutiveMistakes.get(filePath) ?? 0 + 1;
        this.consecutiveMistakes.set(filePath, mistakes);
        const breakerMsg =
```

```
        mistakes >= 3
          ? `\nCIRCUIT BREAKER (3 consecutive diff failures on ${filePath}): Re-read the target lines with 'read' before retrying.`
          : ""
      return {
        success: false,
        output: `SearchReplaceNoExactMatch in ${filePath} near line ${block.startLineHint ?? "unknown"}.${breakerMsg}`
      };
    }

    resolvedSpans.push({
      startIdx: match.startIdx,
      endIdx: match.endIdx,
      replacementLines: replaceLines,
      origSearchFirstLine: searchLines[0] ?? ""
    });
  }

  // 3. Sort spans bottom-to-top so out-of-order LLM blocks apply without shifting earlier indices
  resolvedSpans.sort((a, b) => b.startIdx - a.startIdx);
  for (let i = 0; i < resolvedSpans.length - 1; i++) {
    if (resolvedSpans[i].startIdx < resolvedSpans[i + 1].endIdx) {
      return { success: false, output: `REJECTED: Overlapping SEARCH blocks detected in ${filePath}.` };
    }
  }

  // 4. Record Shadow Git Checkpoint prior to mutation (Roo/Zoo Code architecture)
  const checkpointSha = await shadowGit.recordPreEditCheckpoint(filePath);

  const mutatedLines = [...fileLines];
  for (const span of resolvedSpans) {
    // Restore omitted leading indentation if matched via Pass 3/4 (Aider indentation restoration)
    const origIndent = (fileLines[span.startIdx]?.match(/^\s*/) ?? [""])[0];
    const searchIndent = (span.origSearchFirstLine.match(/^\s*/) ?? [""])[0];
    const indentDelta = origIndent.length > searchIndent.length
      ? origIndent.slice(searchIndent.length) : "";
    const adjustedReplacement = span.replacementLines.map((l) => (l.trim().length > 0 ? indentDelta + l : l));
    mutatedLines.splice(span.startIdx, span.endIdx - span.startIdx, ...adjustedReplacement);
  }
```

```
  const updatedContent = mutatedLines.join("\n");
  await fs.writeFile(filePath, updatedContent, "utf8");

  // 5. Immediate LSP Diagnostic & Syntax Gate (OpenCode + SWE-agent auto-rollback)
  const diagnostics = await lspBus.notifyDidChange(filePath, updatedContent);
  const fatalSyntaxErrors = diagnostics.filter((d) => d.severity === "error" &&
/syntax|parse|expected/i.test(d.message));

  if (fatalSyntaxErrors.length > 0) {
    // SWE-agent Lint-on-Edit Auto-Rollback
    await fs.writeFile(filePath, originalContent, "utf8");
    return {
      success: false,
      output:
        `AUTO-REVERTED (Syntax Error Gate, restored Shadow Checkpoint ${checkpointSha}):\n` +
        fatalSyntaxErrors.map((e) => `  Line ${e.line}: ${e.message}`).join("\n"),
    };
  }

  this.consecutiveMistakes.set(filePath, 0);
  const typeErrorFeedback =
    diagnostics.length > 0
      ? `\n[LSP Diagnostics Feedback (${diagnostics.length} items)]:\n` +
        diagnostics.slice(0, 15).map((d) => `  [${d.severity.toUpperCase()}] Line ${d.line}:
${d.message}`).join("\n")
      : "\n[LSP Diagnostics]: 0 errors.";

  return {
    success: true,
    output: `Applied ${resolvedSpans.length} surgical edit(s) to ${filePath} (Checkpoint
${checkpointSha}).${typeErrorFeedback}`,
  };
}

private findBlockSpan(
  fileLines: string[],
  searchLines: string[],
  startLineHint?: number,
): { startIdx: number; endIdx: number } | null {
  const windowLen = searchLines.length;
  // Stage A: Codex 4-Pass Strict-to-Normalized Sequence Scan (Uniqueness enforced unless
startLineHint disambiguates)
  for (const pass of [1, 2, 3, 4] as const) {
```

```
    const matches: number[] = [];
    for (let i = 0; i <= fileLines.length - windowLen; i++) {
      let ok = true;
      for (let j = 0; j < windowLen; j++) {
        if (normalizeLine(fileLines[i + j], pass) !== normalizeLine(searchLines[j], pass)) {
          ok = false;
          break;
        }
      }
      if (ok) matches.push(i);
    }
    if (matches.length === 1) return { startIdx: matches[0], endIdx: matches[0] + windowLen };
    if (matches.length > 1 && startLineHint !== undefined) {
      matches.sort((a, b) => Math.abs(a + 1 - startLineHint) - Math.abs(b + 1 - startLineHint));
      return { startIdx: matches[0], endIdx: matches[0] + windowLen };
    }
  }

  // Stage B: Roo/Zoo Middle-Out Levenshtein Fuzzy Search within BUFFER_LINES=40 around :start_line:
  if (startLineHint !== undefined) {
    const center = Math.max(0, Math.min(fileLines.length - windowLen, startLineHint - 1));
    const minIdx = Math.max(0, center - this.BUFFER_LINES);
    const maxIdx = Math.min(fileLines.length - windowLen, center + this.BUFFER_LINES);
    const targetNorm = searchLines.map((l) => l.trim()).join("\n");
    let bestScore = 0;
    let bestIdx = -1;

    for (let offset = 0; offset <= this.BUFFER_LINES; offset++) {
      for (const candidate of [center - offset, center + offset]) {
        if (candidate < minIdx || candidate > maxIdx) continue;
        const sliceNorm = fileLines.slice(candidate, candidate + windowLen).map((l) => l.trim()).join("\n");
        const score = levenshteinSimilarity(targetNorm, sliceNorm);
        if (score > bestScore) {
          bestScore = score;
          bestIdx = candidate;
        }
      }
    }
    if (bestScore >= this.FUZZY_THRESHOLD && bestIdx >= 0) {
      return { startIdx: bestIdx, endIdx: bestIdx + windowLen };
    }
```

```
  return null
}
```

Module 3: Personalized PageRank RepoMap Engine (packages/context/src/repomap_engine.py)

This module implements **Aider's Tree-sitter + Personalized PageRank RepoMap** (with 50x active-file weight boost and binary-search token fitting within 15% tolerance)[9].

```python
# packages/context/src/repomap_engine.py
import math
import networkx as nx
from dataclasses import dataclass
from typing import Dict, List, Set, Tuple

@dataclass(frozen=True)
class SymbolTag:
    rel_fname: str
    line: int
    name: str
    kind: str  # "def" | "ref"

def compute_personalized_repomap(
    tags: List[SymbolTag],
    chat_fnames: Set[str],
    mentioned_idents: Set[str],
    max_map_tokens: int = 2048,
) -> List[Tuple[str, str, float]]:
    """
    Implements Aider's Tree-sitter + Personalized PageRank symbol ranking
    with binary-search token fitting within 15% tolerance.
    """
    definers: Dict[str, Set[str]] = {}
    references: Dict[Tuple[str, str], List[str]] = {}

    for t in tags:
        if t.kind == "def":
            definers.setdefault(t.name, set()).add(t.rel_fname)
```

```
        elif t.kind == "ref":
            references.setdefault((t.rel_fname, t.name), []).append(t.rel_fname)

    G = nx.MultiDiGraph()
    for (src_file, ident), ref_list in references.items():
        target_files = definers.get(ident, set())
        if not target_files:
            continue

        # Edge weight heuristics (Aider specification)
        mul = 1.0
        if ident in mentioned_idents:
            mul *= 10.0
        if len(ident) >= 8 and "_" in ident or any(c.isupper() for c in ident[1:]):
            mul *= 10.0
        if ident.startswith("_"):
            mul *= 0.1
        if len(target_files) > 5:
            mul *= 0.1
        if src_file in chat_fnames:
            mul *= 50.0  # Critical 50x boost for references originating in active chat files

        weight = mul * math.sqrt(len(ref_list))
        for dst_file in target_files:
            if dst_file != src_file:
                G.add_edge(src_file, dst_file, weight=weight, ident=ident)

    if G.number_of_nodes() == 0:
        return []

    # Personalization vector biased toward active chat files
    all_nodes = list(G.nodes())
    personalize = {
        node: 100.0 / len(all_nodes) if node in chat_fnames else 1.0 / len(all_nodes)
        for node in all_nodes
    }
    ranked_files = nx.pagerank(G, weight="weight", personalization=personalize,
dangling=personalize)

    # Distribute file PageRank proportionally to (dst_file, symbol_ident) definitions
    ranked_defs: Dict[Tuple[str, str], float] = {}
    for src in G.nodes():
        src_rank = ranked_files.get(src, 0.0)
```

```
        out_edges = list(G.out_edges(src, data=True))
        total_w = sum(d.get("weight", 1) for _, _, d in out_edges)
        if total_w <= 0:
            continue
        for _, dst, data in out_edges:
            if dst not in chat_fnames:  # Exclude files already visible in full in chat
                key = (dst, data.get("ident", ""))
                ranked_defs[key] = ranked_defs.get(key, 0.0) + (src_rank * data.get("weight", 1) / total_w)

    sorted_defs = sorted(
        [(dst, ident, score) for (dst, ident), score in ranked_defs.items()],
        key=lambda x: x[2],
        reverse=True,
    )

    # Binary search token budget fitting (within 15% tolerance of max_map_tokens)
    low, high = 0, len(sorted_defs)
    best_slice = sorted_defs[: min(25, len(sorted_defs))]
    while low <= high:
        mid = (low + high) // 2
        approx_tokens = mid * 18  # ~18 tokens per scope-rendered signature line
        if abs(approx_tokens - max_map_tokens) / max(1, max_map_tokens) < 0.15:
            best_slice = sorted_defs[:mid]
            break
        elif approx_tokens < max_map_tokens:
            best_slice = sorted_defs[:mid]
            low = mid + 1
        else:
            high = mid - 1

    return best_slice
```

Module 4: AST-Gated OS Kernel Sandbox & Fixed-Point Wrapper Stripper (packages/sandbox/src/kernel-sandbox.ts)

This module combines **OpenAI Codex CLI's parameterized Seatbelt SBPL profile** (protecting .git and .omni even inside writable workspaces) and **Bubblewrap + Landlock + Seccomp** with **Claude Code's fixed-point wrapper stripper and SOCKS5 domain proxy**[28].

TypeScript

```ts
// packages/sandbox/src/kernel-sandbox.ts
import * as path from "node:path";

export interface SandboxPolicyConfig {
  mode: "read-only" | "workspace-write";
  workspaceRoot: string;
  extraWritableRoots: string[];
  allowedDomains: string[];
  allowUnsandboxedCommands: false; // Strict Fail-Closed Mode by default
  socksProxyPort?: number;
}

// Claude Code shouldUseSandbox.ts Fixed-Point Command Wrapper Stripper
export function extractBaseCommandFixedPoint(rawCmd: string): string {
  let current = rawCmd.trim();
  const wrapperRegex =
/^(?:(?:[A-Z_][A-Z0-9_]*=\S+\s+)|(?:sudo|env|nice|nohup|time|command|timeout\s+\S+)\s+)+/i;
  let prev = "";
  while (current !== prev) {
    prev = current;
    current = current.replace(wrapperRegex, "").trim();
  }
  return current.split(/\s+/)[0] ?? "";
}

// Generate macOS Seatbelt SBPL Profile (Codex + Claude Code hybrid)
// Explicitly protects .git/ and .omni/ from tampering even in workspace-write mode
export function compileMacosSeatbeltProfile(cfg: SandboxPolicyConfig): {
  executable: "/usr/bin/sandbox-exec";
  args: string[];
} {
  const gitDir = path.join(cfg.workspaceRoot, ".git");
  const omniDir = path.join(cfg.workspaceRoot, ".omni");

  const sbpl = [
    "(version 1)",
    "(deny default)",
    "(allow process-exec)",
    "(allow process-fork)",
    "(allow signal (target same-sandbox))",
    "(allow user-preference-read)",
    "(allow process-info* (target same-sandbox))",
    "(allow ipc-posix-sem)",
```

```
  "(allow pseudo-tty)"
  '(allow file-read* (subpath "/"))'
  // Block reading credential stores by default (Claude Code sandbox.credentials.files)
  '(deny file-read* (subpath (param "SSH_DIR")) (subpath (param "AWS_DIR")))'
  '(allow file-write-data (require-all (path "/dev/null") (vnode-type CHARACTER-DEVICE)))'
]

if (cfg.mode === "workspace-write") {
  sbpl.push(`
  (allow file-write*
    (require-all
      (subpath (param "WRITABLE_ROOT_0"))
      (require-not (subpath (param "WRITABLE_ROOT_0_RO_GIT")))
      (require-not (subpath (param "WRITABLE_ROOT_0_RO_OMNI")))
    )
    (subpath "/private/tmp")
  )
  `);
}

if (cfg.socksProxyPort) {
  sbpl.push(`(allow network-outbound (remote ip "localhost:${cfg.socksProxyPort}"))`);
}

return {
  executable: "/usr/bin/sandbox-exec",
  args: [
    "-p",
    sbpl.join("\n"),
    `-DWRITABLE_ROOT_0=${cfg.workspaceRoot}`,
    `-DWRITABLE_ROOT_0_RO_GIT=${gitDir}`,
    `-DWRITABLE_ROOT_0_RO_OMNI=${omniDir}`,
    `-DSSH_DIR=${path.join(process.env.HOME ?? "", ".ssh")}`,
    `-DAWS_DIR=${path.join(process.env.HOME ?? "", ".aws")}`,
    "--",
  ],
};
}

// Generate Linux Bubblewrap (bwrap) Isolation Args
export function compileLinuxBwrapArgs(cfg: SandboxPolicyConfig): {
  executable: "bwrap",
  args: string[],
} {
```

```
const gitDir = path.join(cfg.workspaceRoot, ".git")
const omniDir = path.join(cfg.workspaceRoot, ".omni")
const args = [
  "--unshare-all",
  "--die-with-parent",
  "--ro-bind", "/", "/",
  "--dev", "/dev",
  "--proc", "/proc",
  "--tmpfs", "/tmp",
]
if (cfg.mode === "workspace-write") {
  args.push("--bind", cfg.workspaceRoot, cfg.workspaceRoot)
  // Re-bind .git and .omni as read-only inside the writable workspace mount
  args.push("--ro-bind-try", gitDir, gitDir)
  args.push("--ro-bind-try", omniDir, omniDir)
}
args.push("--")
return { executable: "bwrap", args }
}
```

Module 5: Harness² Recursive Self-Evolution Engine (packages/evolver/src/propose-probe-compose.ts)

This module implements **Google Research's Harness² (Propose–Probe–Compose)** loop (R = 4k rollouts, P = 3k improver sessions, 3 fail-closed validation gates) and **Nous Hermes Agent's Skill Crystallizer** for omni evolve[14].

```
TypeScript
// packages/evolver/src/propose-probe-compose.ts
export type EditableSurfaceId = "c1" | "c2" | "c3" | "c4" | "c5" | "c6" | "c7" | "c8"

export interface HarnessCandidateTree {
  id: string
  isDomainSpecific: boolean
  targetSurface: EditableSurfaceId
  componentFiles: Record<string, string> // Static text only; never executed by improver
}

export interface RolloutEvidence {
  candidateId: string
```

```
  deliveredExists: boolean
  lspErrorsRemaining: number
  testExitCode: number
  turnCount: number
  tokenSpendUsd: number
  trajectoryDigest: string
}

export class Harness2Evolver {
  private readonly LEAK_MARKERS = ["RUBRICS.json", "score.json", "PASS_TO_PASS", "FAIL_TO_PASS"];

  // Three Fail-Closed Validation Gates (Harness² validate_harness.py spec)
  public validateTreeFailClosed(tree: HarnessCandidateTree, alwaysOnTokenCap = 900): string[] {
    const errors: string[] = [];
    const c1Prompt = tree.componentFiles[".omni/harness/c1-system-prompt.md"] ?? "";
    const approxTokens = Math.ceil(c1Prompt.length / 4);

    // Gate 1: Core Structural & Token Cap Gate (Prevents Harness Tax regression!)
    if (approxTokens > alwaysOnTokenCap) {
      errors.push(
        `OVER_CAP_ALWAYS_ON_TEXT: c1-system-prompt is ~${approxTokens} tokens (max ${alwaysOnTokenCap}). Move domain prose to a lazy c6 SKILL.md.`
      );
    }

    // Gate 2 & Gate 3: Domain Invariants & Grader-Verdict Leak Tripwire
    for (const [filePath, content] of Object.entries(tree.componentFiles)) {
      for (const marker of this.LEAK_MARKERS) {
        if (content.includes(marker)) {
          errors.push(`GRADER_VERDICT_LEAK_TRIPWIRE: Forbidden rubric marker '${marker}' found in ${filePath}.`);
        }
      }
      if (
        filePath.endsWith("SKILL.md") &&
        !/^---\n[\s\S]*?name:\s*\S+[\s\S]*?description:\s*\S+[\s\S]*?\n---/.test(content)
      ) {
        errors.push(`SKILL_FRONTMATTER_MISMATCH: ${filePath} is missing required YAML name/description frontmatter.`);
      }
    }
```

```
  return errors;
}

// Contrastive Propose-Probe-Compose Step (No ground-truth verifier required)
public async runProposeProbeCompose(params: {
  taskPrompt: string;
  baseTree: HarnessCandidateTree;
  kBudget: number;
  mode: "parallel" | "sequential";
  frozenSurfaces: EditableSurfaceId[];
  capUsd: number;
  estimatedUsd: number;
  proposeFn: (base: HarnessCandidateTree, k: number) => Promise<HarnessCandidateTree[]>;
  probeFn: (candidate: HarnessCandidateTree, task: string) => Promise<RolloutEvidence>;
  composeFn: (contrasts: RolloutEvidence[], candidates: HarnessCandidateTree[]) => Promise<{
    adopted: HarnessCandidateTree;
    lesson: { rule: string; evidence: string; scope: string; invalidationTrigger: string };
  }>;
}) {
  // Fail-closed spend cap guard
  if (params.estimatedUsd <= 0 || params.estimatedUsd >= params.capUsd) {
    throw new Error(
      `SPEND_CAP_GUARD: Estimated spend ($${params.estimatedUsd}) must be > 0 and strictly below --cap-usd ($${params.capUsd}).`
    );
  }

  const baseErrors = this.validateTreeFailClosed(params.baseTree);
  if (baseErrors.length > 0) throw new Error(`INVALID_BASE_TREE:\n${baseErrors.join("\n")}`);

  let currentTree = params.baseTree;
  const lessonsKept: Array<{ rule: string; evidence: string; scope: string; invalidationTrigger: string }> = [];

  const steps = params.mode === "sequential" ? params.kBudget : 1;
  for (let step = 0; step < steps; step++) {
    // 1. PROPOSE (Freeze all surfaces except target surface to keep evidence readable)
    const candidates = await params.proposeFn(currentTree, params.mode === "parallel" ? params.kBudget : 1);
    for (const c of candidates) {
      const errs = this.validateTreeFailClosed(c);
      if (errs.length > 0) throw new Error(`CANDIDATE_VALIDATION_FAILED (${c.id}):\n${errs.join("\n")}`);
    }
```

```
    // 2. PROBE (Execute task rollouts under candidate harnesses and collect behavioral evidence)
    const evidences: RolloutEvidence[] = [];
    for (const candidate of [currentTree, ...candidates]) {
      evidences.push(await params.probeFn(candidate, params.taskPrompt));
    }

    // 3. COMPOSE (Synthesize refined harness from behavioral contrasts without ground-truth rubrics)
    const { adopted, lesson } = await params.composeFn(evidences, [currentTree, ...candidates]);
    const adoptedErrors = this.validateTreeFailClosed(adopted);
    if (adoptedErrors.length > 0) throw new Error(`COMPOSED_TREE_INVALID:\n${adoptedErrors.join("\n")}`);

    currentTree = adopted;
    lessonsKept.push(lesson);
  }

  return {
    adoptedTree: currentTree,
    lessonsKept,
    resourceLedger: {
      expectedRolloutsR: 4 * params.kBudget,
      expectedImproverSessionsP: 3 * params.kBudget,
    },
  };
}
}
```

3.4 Unified CLI Command Reference (omni)

OmniHarness exposes a single CLI binary (omni) that unifies interactive terminal workflows, IDE ACP integration, shared team workspaces, Kiro-style spec engineering, and Harness² self-improvement:

Command / Interactive Slash Action	Operational Behavior & Architectural Provenance
omni	Launches the differential-rendering scrollback TUI (*Pi* + *Charmbracelet Crush*) with default-on OS kernel sandboxing

	(Seatbelt on macOS, bwrap + Landlock + seccomp on Linux) and on-demand LSP attachment[29].
omni init [--sync-targets claudecode,cursor,codex,zoocode]	Scaffolds .omni/harness/c1..c8, initializes the Shadow Git checkpoint repository, builds the Tree-sitter + PageRank RepoMap index, and compiles cross-tool configs via rulesync[9].
omni spec "<feature prompt>"	Runs **Kiro + Factory Droid Spec Mode**: generates requirements.md, design.md, and tasks.md using a low-cost spec model (--spec-model), checks requirements for contradictions, and pauses for human review[48].
omni exec [--auto low|medium|high] [--worktree <name>] [--mission] "<task>"	Headless non-interactive execution for CI/CD pipelines, supporting isolated git worktrees (--worktree) and Worker/Validator model splitting (--mission)[48].
omni acp	Starts the **Agent Client Protocol (ACP)** JSON-RPC 2.0 server over stdio so Zed, JetBrains IDEs, VS Code, and Neovim can drive OmniHarness natively[65].
omni serve [--port 4096]	Starts the **OpenCode / Crush shared workspace HTTP+SSE daemon**, allowing mobile, web, and secondary TUI clients to attach live to the same session[32].
omni evolve --surface c6 --k 2 --mode parallel --cap-usd 5.00	Runs **Google Research's Harness² Propose–Probe–Compose loop** on a single unfrozen component surface (c1–c8), validates candidates against the 900-token Turn-1 cap and leak tripwires, and outputs a diff summary of the evolved harness and extracted SKILL.md playbooks for human approval[14].

/tree, /fork, /clone, /undo	Interactive slash commands to navigate the append-only JSONL session DAG (*Pi*), run MCTS branch exploration (*Moatless*), or restore any Shadow Git checkpoint (*Roo/Zoo*)[37].
/subtask <prompt> & /oracle <question>	/subtask spawns a prompt-cache-sharing fork subagent (*Claude Code*); /oracle escalates architectural/debugging analysis to the read-only frontier reasoning subagent (*Sourcegraph Amp*)[45].
/compact [focus] & /creator <description>	/compact runs probe-verified context compaction followed by the 6-step rehydration sequence (*Gemini CLI + Claude Code*); /creator synthesizes and hot-mounts a new Cordis plugin or SKILL.md into the live session via ctx.effect() (*DeepSeek Harness*)[2].

Works cited

1. Inside the Scaffold: A Source-Code Taxonomy of Coding Agent, https://arxiv.org/html/2604.03515v2
2. https://github.com/deusyu/harness-engineering/blob/main/works/inside-the-scaffold-paper-translation.md
3. https://terminaltrove.com/compare/ai-coding-agents/
4. Pi Coding Agent: Architecture, Agent Loop, Extension System, https://xiaow.dev/claude_notes/2026-03-27---Research---Pi-Coding-Agent---Agent-Loop,-Extension-and-Plugin-System
5. https://venturebeat.com/orchestration/a-coding-agents-harness-can-triple-its-cost-without-making-it-more-accurate-study-finds
6. Google Harness2: The Framework That Lets an Agent Improve Its, https://medium.com/data-science-in-your-pocket/google-harness2-the-framework-that-lets-an-agent-improve-its-own-playbook-ce4b8ba157a7
7. Agent-Computer Interfaces Enable Automated Software Engineering, https://neurips.cc/virtual/2024/poster/93753
8. https://cefboud.com/posts/coding-agents-internals-opencode-deepdive/
9. PageRank Repo Map — Automatic Codebase Context Selection via, https://github.com/NousResearch/hermes-agent/issues/535
10. Google Research Harness²: Recursive AI Agent Improvement, https://www.datastudios.org/post/google-research-harness2-recursive-ai-agent-improvement-open-source
11. apply_diff | Roo Code Documentation - GitHub Pages,

https://roocodeinc.github.io/Roo-Code/advanced-usage/available-tools/apply-diff
12. SWE-bench Leaderboards, https://www.swebench.com/
13. https://dbggr.dev/articles/ai-coding-agents-compared/
14. GitHub - google-research/harness2, https://github.com/google-research/harness2
15. kenhuangus/harness2-skill: Muse skill for Google's Harness², https://github.com/kenhuangus/harness2-skill
16. What Is DeepSeek Harness? The Open-Source Coding Agent, https://www.mindstudio.ai/blog/deepseek-harness-coding-agent
17. deepseek-harness/AGENTS.md at master - GitHub, https://github.com/deepseek-ai/deepseek-harness/blob/master/AGENTS.md
18. What Is DeepSeek Harness? The Agent Where Everything Is a Plugin, https://www.datacamp.com/blog/what-is-deepseek-harness
19. deepseek-harness/docs/architecture.md at master - GitHub, https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/architecture.md
20. https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/subsystems/README.md
21. Hermes Agent GitHub Repo: Architecture and Contributing Guide, https://fast.io/resources/hermes-agent-github-open-source/
22. hermes-agent/website/docs/developer-guide/architecture.md at main, https://github.com/NousResearch/hermes-agent/blob/main/website/docs/developer-guide/architecture.md
23. Deep Agents: Open Source Agent Harness | LangChain, https://www.langchain.com/deep-agents
24. The Empire, Long Divided, Must Unite: Architectural Convergence in, https://arxiv.org/html/2608.23953v1
25. Inside Claude Code's Compaction System, https://decodeclaude.com/compaction-deep-dive/
26. Claude Code Subagents and Multi-Agent Orchestration Guide, https://hidekazu-konishi.com/entry/claude_code_subagents_and_orchestration_guide.html
27. https://github.com/Piebald-AI/claude-code-system-prompts
28. Shell & Execution Tools | SinghCoder/claude-code - DeepWiki, https://deepwiki.com/SinghCoder/claude-code/3.2-shell-and-execution-tools
29. OpenAI Codex CLI -- Sandbox Analysis Report | Agent Safehouse, https://agent-safehouse.dev/docs/agent-investigations/codex
30. Overview | openai/codex - Zread, https://zread.ai/openai/codex
31. Exact-Match Edits vs Patches: How Claude Code, Codex and Aider, https://dev.to/gabrielanhaia/exact-match-edits-vs-patches-how-claude-code-codex-and-aider-write-to-your-files-45d1
32. What Is OpenCode? The Open-Source AI Coding Agent Explained, https://www.datacamp.com/blog/what-is-opencode
33. OpenCode Review: Open Source AI Coding Agent, 2 Weeks In,

https://www.heyuan110.com/posts/ai/2026-03-13-opencode-ai-coding-agent-review/
34. LSP Servers - OpenCode, https://opencode.ai/docs/lsp/
35. https://github.com/charmbracelet/crush
36. Pi — Anatomy of a minimal coding agent powering OpenClaw, https://shivamagarwal7.medium.com/agentic-ai-pi-anatomy-of-a-minimal-coding-agent-powering-openclaw-5ecd4dd6b440
37. pi Deep Dive: The Minimal Architecture Behind 95,000 GitHub Stars, https://www.developersdigest.tech/blog/pi-deep-dive-agent-toolkit-architecture
38. https://aider.chat/docs/more/edit-formats.html
39. Edit Format Implementations | Aider-AI/aider - DeepWiki, https://deepwiki.com/Aider-AI/aider/3.1-edit-format-implementations
40. Gemini CLI Guide 2026: Commands, GEMINI.md & Automation, https://lucaberton.com/blog/gemini-cli-complete-guide-commands-automation-2026/
41. Sandboxing in Gemini CLI, https://geminicli.com/docs/cli/sandbox/
42. Google Antigravity CLI: Orchestrating Parallel AI Agents - DataCamp, https://www.datacamp.com/tutorial/antigravity-cli
43. Spec-Driven Development with Antigravity CLI — Structured Agent, https://codelabs.developers.google.com/sdd-agy-cli
44. Getting Started with Google Antigravity - Codelabs, https://codelabs.developers.google.com/getting-started-google-antigravity
45. Oracle - Amp Code, https://ampcode.com/news/oracle
46. Amp: Coding agent and dev environment built for the frontier, https://ampcode.com/
47. Custom droids (subagents) - Factory docs, https://docs.factory.com/harness/subagents
48. Factory AI and the Model Routing Era: How Coding Agents Are, https://www.developersdigest.tech/blog/factory-ai-droid-model-routing-costs
49. Plandex | Open Source AI Coding Agent for Large Codebases, https://railway.com/deploy/plandex-open-source-ai-coding-agent-for-large-codebases--plandex
50. GitHub - Rubentxu/mentat: Agentic CI/CD platform in Rust, https://github.com/Rubentxu/mentat
51. Agents - Goose Documentation, https://block-goose.mintlify.app/concepts/agents
52. goose | Your open source AI agent, https://goose-docs.ai/
53. Zoo Code gives you a whole dev team of AI agents in your code editor., https://github.com/Zoo-Code-Org/Zoo-Code
54. Add Zoo Code to the third-party coding agent API whitelist #2416, https://github.com/MoonshotAI/kimi-cli/issues/2416
55. Boomerang Tasks: Orchestrate Complex Workflows | Roo Code, https://roocodeinc.github.io/Roo-Code/features/boomerang-tasks/
56. Checkpoints | Roo Code Documentation - GitHub Pages, https://roocodeinc.github.io/Roo-Code/features/checkpoints/

57. Kilo Code Alternatives — Cursor, Copilot, Windsurf & More Compared, https://kilo.ai/kilo-code/alternatives
58. Quick Start - Continue Docs, https://docs.continue.dev/ide-extensions/agent/quick-start
59. Agentic IDE Comparison: Cursor vs Windsurf vs Antigravity, https://www.codecademy.com/article/agentic-ide-comparison-cursor-vs-windsurf-vs-antigravity
60. Kiro IDE: AWS Built an Agentic IDE That Writes Code from Specs, https://www.paperclipped.de/en/blog/kiro-ide-spec-driven-development/
61. AWS Kiro: The Agentic IDE That Makes Specs the Unit of Work, https://dzone.com/articles/kiro-feature-to-requirements-design-tasks
62. GitHub - freeCodeCamp/agent-scaffold: Make coding agents less, https://github.com/freeCodeCamp/agent-scaffold
63. GitHub - dyoshikawa/rulesync: A Utility CLI for AI Coding Agents, https://github.com/dyoshikawa/rulesync
64. https://github.com/soumatheusgomes/vibe-coding-toolkit
65. Agent Client Protocol (ACP) Explained: ACP vs MCP, Editor ... - Morph, https://www.morphllm.com/agent-client-protocol
66. Agent Client Protocol (ACP): The LSP for AI Coding Agents Explained, https://blog.marcnuri.com/agent-client-protocol-acp-introduction
67. SWE-agent — Deep Dive & Build-Your-Own Guide - DEV Community, https://dev.to/truongpx396/swe-agent-deep-dive-build-your-own-guide-ade
68. mini-swe-agent: a 100 line agent on bash alone | Hysen Labs, https://hysenlabs.com/projects/swe-agent-mini-swe-agent
69. GitHub - SWE-agent/mini-swe-agent: The 100 line AI agent that, https://github.com/swe-agent/mini-swe-agent
70. OpenHands: The Open-Source AI Coding Agent Explained, https://www.datacamp.com/tutorial/openhands
71. API-based Sandbox - OpenHands Docs, https://docs.openhands.dev/sdk/guides/agent-server/api-sandbox
72. OpenHands/docs/architecture.md at main - GitHub, https://github.com/OpenHands/OpenHands/blob/main/docs/architecture.md
73. Agentless: Demystifying LLM-based Software Engineering Agents, https://www.alphaxiv.org/abs/2407.01489
74. AutoCodeRover: Autonomous Program Improvement - arXiv, https://arxiv.org/pdf/2404.05427
75. GitHub - zanwenfu/auto-code-rover, https://github.com/zanwenfu/auto-code-rover
76. SWE-AF/docs/ARCHITECTURE.md at main - Agent-Field - GitHub, https://github.com/Agent-Field/SWE-AF/blob/main/docs/ARCHITECTURE.md
77. SWE-Search: Enhancing Software Agents with Monte Carlo Tree, https://www.alphaxiv.org/gu/abs/2410.20285
78. Improving Diff Edits by 10% | Cline Blog, https://cline.bot/blog/improving-diff-edits-by-10
79. claude-code-ultimate-guide/guide/security/sandbox-native.md at main,

https://github.com/FlorianBruniaux/claude-code-ultimate-guide/blob/main/guide/security/sandbox-native.md
80. Claude Code Subagents: What They Are and How to Build One, https://alexcloudstar.com/blog/claude-code-subagents-guide-2026/
81. Overview | charmbracelet/bubbletea - Zread, https://zread.ai/charmbracelet/bubbletea
82. Droid (Factory CLI) -- Sandbox Analysis Report - Agent Safehouse, https://agent-safehouse.dev/docs/agent-investigations/droid
83. Models used by Sourcegraph - Sourcegraph docs, https://sourcegraph.com/docs/ai/models
84. How to use subagents in AI coding with Amp - Medium, https://medium.com/@matthewtanner91/how-to-use-subagents-in-ai-coding-with-amp-8b8418486782
85. goose Docs Extension | goose | Your open source AI agent, https://goose-docs.ai/docs/mcp/goose-docs-mcp/
86. Open source AI coding agent. Designed for large projects and real, https://github.com/plandex-ai/plandex
87. Tools - Cline documentation, https://docs.cline.bot/tools-reference/all-cline-tools
88. Automate your development workflow with Kiro's AI agent hooks, https://kiro.dev/blog/automate-your-development-workflow-with-agent-hooks/
89. Kiro: Move beyond AI coding to agentic engineering, https://kiro.dev/
90. Overview - mini-SWE-agent documentation, https://mini-swe-agent.com/
91. OpenHands: An Open Platform for AI Software Developers as, https://openreview.net/forum?id=OJd3ayDDoF
92. README_swebench.md - OpenAutoCoder/Agentless - GitHub, https://github.com/OpenAutoCoder/Agentless/blob/main/README_swebench.md
93. Coding Agents - Open Source Approaches on SWE-Bench - Medium, https://medium.com/@te2be/coding-agents-open-source-approaches-on-swe-bench-074cc28c5bb0
94. SWE-Search: Enhancing Software Agents with Monte Carlo Tree, https://arxiv.org/html/2410.20285v1
95. Harness2 by Google Research – Recursive Agent Harnessing, https://news.ycombinator.com/item?id=50151059
96. Harness^2: Recursive Agent Harnessing for an Open World - arXiv, https://arxiv.org/abs/2610.14224
97. vibe-coding-toolkit/docs/tools/23-harness2.md at main - GitHub, https://github.com/soumatheusgomes/vibe-coding-toolkit/blob/main/docs/tools/23-harness2.md
98. harness2/docs/BENCHMARKS.md at main - GitHub, https://github.com/google-research/harness2/blob/main/docs/BENCHMARKS.md
99. DeepSeek Harness: Everything is a Plugin. - GitHub, https://github.com/deepseek-ai/deepseek-harness
100. Inside DeepSeek Harness: Cordis, Session Events, Tool Pipelines, https://habr.com/en/articles/1070958/
101. DeepSeek Harness | Explore the limits of intelligence,

https://www.deepseek.com/en/harness/
102. DeepSeek Harness Opens the Agent Run Loop to Plugins, https://developer.tenten.co/deepseek-harness-plugin-runtime-guide
103. NousResearch/hermes-agent: The agent that grows with you - GitHub, https://github.com/nousresearch/hermes-agent
104. Hermes Agent — Open-Source AI Agent That ... - Nous Research, https://hermes-agent.nousresearch.com/
105. Here is Claude Code's `/compact` Prompt : r/ClaudeAI - Reddit, https://www.reddit.com/r/ClaudeAI/comments/1jr52qj/here_is_claude_codes_compact_prompt/
106. Claude Code subagents: the 2026 production playbook ... - Totalum, https://www.totalum.app/blog/claude-code-subagents-totalum
107. Choose a sandbox environment - Claude Code Docs, https://code.claude.com/docs/en/sandbox-environments
108. Codex Sandbox: Modes, Limits, Safe Setups (2026) - Agent 37, https://www.agent37.com/blog/codex-sandbox
109. Codex CLI Sandboxing: Seatbelt, Bubblewrap, Landlock, and Docker, https://blog.checo.cc/en/posts/AI/9
110. Codex, Behind the Harness — Dominik Kundel, OpenAI - daily.dev, https://daily.dev/posts/codex-behind-the-harness-dominik-kundel-openai-aheps4agd
111. Terminal User Interfaces: Review of Crush (Ex-OpenCode AI), https://thenewstack.io/terminal-user-interfaces-review-of-crush-ex-opencode-ai/
112. https://medium.com/@falexm/inside-opencode-understanding-the-architecture-behind-the-ai-runtime-01236d9370ff
113. pi (pi.dev) - AI Agent Architectures, https://agentarchitectures.com/framework/pi-dev
114. mariozechner/pi-coding-agent - NPM, https://www.npmjs.com/package/@mariozechner/pi-coding-agent
115. Repository map - Aider, https://aider.chat/docs/repomap.html
116. Unified diffs make GPT-4 Turbo 3X less lazy - Aider, https://aider.chat/docs/unified-diffs.html
117. Enhancement: Consider fuzzy matching of the SEARCH pattern. #306, https://github.com/paul-gauthier/aider/issues/306
118. Bitloops for Gemini CLI — Context Engineering for Google's AI, https://bitloops.com/gemini
119. Google Gemini CLI Cheatsheet - Ollayor's Blog, https://www.ollayor.uz/blog/google-gemini-cli-cheatsheet
120. Google Antigravity CLI, https://antigravity.google/blog/introducing-google-antigravity-cli
121. What is agentic coding? How it works and use cases | Google Cloud, https://cloud.google.com/discover/what-is-agentic-coding
122. Factory Droid Tutorial: Missions, Skills, and Agents - Sid Bharath, https://sidbharath.com/blog/factory-ai-guide/

123. droid.md - claude-code-templates - GitHub, https://github.com/davila7/claude-code-templates/blob/main/cli-tool/components/agents/expert-advisors/droid.md
124. How to use Amp's Oracle to plan complex refactoring tasks, https://dev.to/matt_tanner_f9c36595644ad/how-to-use-amps-oracle-to-plan-complex-refactoring-tasks-4a17
125. onecloner/mentat - The AI Coding Assistant - GitHub, https://github.com/onecloner/mentat
126. What Is Roo Code? The Archived VS Code Agent, Explained, https://munderdifflin/blog/what-is-roo-code/
127. The Full Story Behind the Rebirth of a 3 Million User AI Coding Tool, https://note.com/snake_dragon/n/n104c4a3dc11c?hl=en
128. Roo Code 3.8 - Boomerang Tasks, Smarter Diff Edits, Multi-Window, https://www.reddit.com/r/ChatGPTCoding/comments/1j62zn0/roo_code_38_boomerang_tasks_smarter_diff_edits/
129. Cline and Roo Code 'Without Required Parameter' Error Fixed, https://www.swytchcode.com/content/cline-roo-code-missing-required-parameter
130. From spec to production: a three-week drug discovery agent using Kiro, https://aws.amazon.com/blogs/industries/from-spec-to-production-a-three-week-drug-discovery-agent-using-kiro/
131. Coding-Agent Harnesses - dipakkr/ai-engineering-guide - GitHub, https://github.com/dipakkr/ai-engineering-guide/blob/main/08-agent-harnesses-and-skills/04-coding-agent-harnesses.md
132. Kilo Code vs Windsurf: Which AI Agent Is Better? - Standard Compute, https://standardcompute.com/best-ai-agent/kilo-code-vs-windsurf
133. Continue.dev: Open-Source AI Code Agent Guide - Better Stack, https://betterstack.com/community/guides/ai/continue-dev-ai/
134. Zed — Agent Client Protocol, https://zed.dev/acp
135. Agent Client Protocol: Introduction, https://agentclientprotocol.com/get-started/introduction
136. deepseek-harness-plugin · GitHub Topics, https://github.com/topics/deepseek-harness-plugin?l=typescript&o=desc&s=updated
137. Mini-SWE-agent - Learn AI - Miraheze, https://ai.miraheze.org/wiki/Mini-SWE-agent
138. Agent Computer Interface (ACI) - SWE-agent, https://swe-agent.com/0.7/background/aci/
139. SWE-agent — Knowledge - Notes, https://notes.helloworld.co.in/term/swe-agent-5697
140. Architecture - SWE-agent documentation, https://swe-agent.com/latest/background/architecture/
141. We (the Princeton SWE-bench team) built an agent in - Hacker News, https://news.ycombinator.com/item?id=45001234
142. OpenHands | Open Source AI Coding Agent Platform,

https://www.openhands.dev/
143. SWE-bench Verified (Agentless) Leaderboard - LLM Stats, https://llm-stats.com/benchmarks/swe-bench-verified-(agentless)
144. an agentless approach to automatically solve software development, https://github.com/openautocoder/agentless
145. AutoCodeRover: Autonomous Program Improvement - Clio AI Labs, https://www.clioapp.ai/research/autocoderover
146. AutoCodeRover: AST-Aware Program Improvement - EulerFold, https://www.eulerfold.com/research-decoded/autocoderover-ast-software-engineering-agent
147. DeepSeek Harness vs Pi Agent | AgentsPulse, https://agentspulse.github.io/tutorials/deepseek-harness-vs-pi-agent/